

SOLIDWORKS API (.NET VBA C#) — Examples (38/50): **Insert_Camera_Example_CSharp-Insert_GIof_Example_VB**

SOLIDWORKS API Documentation (converted from CHM)

Insert Camera Example (C#)

This example shows how to insert a camera and print out its settings.

```
//-----
// Preconditions:
// 1. Open a model document that does not have any cameras inserted.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a camera in the model document.
// 2. Click View > Lights and Cameras > Properties > Camera1.
// 3. Click Zoom to Fit.
// 4. Examine the Immediate window, the graphics area, and the
//    Camera1 PropertyManager page.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Camera swCamera = default(Camera);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Insert a camera
            swCamera = (Camera)swModelDocExt.InsertCamera();

            // Print out camera settings
            Debug.Print("ID" = " + swCamera.ID);
            Debug.Print("Focal distance" = " + swCamera.GetFocalDistance() * 1000.0 + " mm");
            Debug.Print("Perspective mode" = " + swCamera.Perspective);
            Debug.Print("Depth of field effects enabled" = " + swCamera.DepthOfFieldEnabled);
            Debug.Print("Distance from focal plane to where focus is lost" = " + swCamera.DepthOfFieldOffset * 1000.0 + " mm");
            Debug.Print("Horizontal angle of the field of view" = " + swCamera.FieldOfViewAngle * 1000.0 + " mm");
            Debug.Print("Depth of the field of view" = " + swCamera.FieldOfViewDepth * 1000.0 + " mm");
            Debug.Print("Height of the field of view" = " + swCamera.FieldOfViewHeight * 1000.0 + " mm");

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Camera Example (VBA)

This example shows how to insert a camera and print out its settings.

```
'-----  
' Preconditions:  
' 1. Open a model document that does not have any cameras inserted.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Inserts a camera in the model document.  
' 2. Click View > Lights and Cameras > Properties > Camera1.  
' 3. Click Zoom to Fit.  
' 4. Examine the Immediate window, the graphics area, and the  
'    Camera1 PropertyManager page.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swCamera As SldWorks.Camera
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension
```

```
    ' Insert a camera  
    Set swCamera = swModelDocExt.InsertCamera
```

```
    ' Print out camera settings
```

```
    Debug.Print "ID" = " & swCamera.ID  
    Debug.Print "Focal distance" = " & swCamera.GetFocalDistance * 1000# & " mm"  
    Debug.Print "Perspective mode" = " & swCamera.Perspective  
    Debug.Print "Depth of field effects enabled" = " & swCamera.DepthOfFieldEnabled  
    Debug.Print "Distance from focal plane to where focus is lost" = " & swCamera.DepthOfFieldOffset * 1000# & " mm"  
    Debug.Print "Horizontal angle of the field of view" = " & swCamera.FieldOfViewAngle * 1000# & " mm"  
    Debug.Print "Depth of the field of view" = " & swCamera.FieldOfViewDepth * 1000# & " mm"  
    Debug.Print "Height of the field of view" = " & swCamera.FieldOfViewHeight * 1000# & " mm"
```

End Sub

Insert Camera Example (VB.NET)

This example shows how to insert a camera and print out its settings.

```
'-----
' Preconditions:
' 1. Open a model document that does not have any cameras inserted.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a camera in the model document.
' 2. Click View > Lights and Cameras > Properties > Camera1.
' 3. Click Zoom to Fit.
' 4. Examine the Immediate window, the graphics area, and the
'    Camera1 PropertyManager page.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swCamera As Camera

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Insert a camera
        swCamera = swModelDocExt.InsertCamera

        ' Print out camera settings
        Debug.Print("ID" & swCamera.ID)
        Debug.Print("Focal distance" & swCamera.GetFocalDistance * 1000.0# & " mm")
        Debug.Print("Perspective mode" & swCamera.Perspective)
        Debug.Print("Depth of field effects enabled" & swCamera.DepthOfFieldEnabled)
        Debug.Print("Distance from focal plane to where focus is lost" & swCamera.DepthOfFieldOffset * 1000.0# & " mm")
        Debug.Print("Horizontal angle of the field of view" & swCamera.FieldOfViewAngle * 1000.0# & " mm")
        Debug.Print("Depth of the field of view" & swCamera.FieldOfViewDepth * 1000.0# & " mm")
        Debug.Print("Height of the field of view" & swCamera.FieldOfViewHeight * 1000.0# & " mm")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Insert Cavity Example (C#)

This example shows how to insert a cavity in a mold base.

```
//-----
// Preconditions:
// 1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt
//    to that folder.
// 2. Click File > New > Part > OK to create a new part document.
//    a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
//    b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
//    c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
// 3. Click File > New > Assembly > OK to create a new assembly document.
//    a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
//    b. Click OK.
//    c. Click Insert Components on the Assembly toolbar, click Browse in
//       Part/Assembly to Insert, click telephone.sldprt located in the folder
//       created in step 1, and click Open.
//    d. Drop telephone.sldprt in the graphics area.
//    e. Click View > Display > Hidden Lines Visible.
//    f. In the FeatureManager design tree, click telephone<1> and click Move Component
//       in the Assembly toolbar.
//    g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
//       TIP: Change the view orientation to Top and Front to help center telephone<1>.
//    h. Click File > Save All > Rebuild and save the document (recommended),
//       navigate to the folder created in step 1, type AssemInterim.sldasm in
//       File name, and click Save.
// 4. Click telephoneMoldBase<1> in the FeatureManager design tree.
// 5. Ctrl+click telephone<1> in the FeatureManager design tree.
//-----
// Postconditions:
// 1. Creates Cavity1 in telephoneMoldBase<1>.
// 2. Expand telephoneMoldBase<1> to verify step 1.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            AssemblyDoc swAssy = default(AssemblyDoc);
            Component2 swMoldBaseComp = default(Component2);
            Component2 swCoreComp1 = default(Component2);
            int nRetVal = 0;
            int nInfo = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swMoldBaseComp = (Component2)swSelMgr.GetSelectedObjectsComponent2(1);
            swCoreComp1 = (Component2)swSelMgr.GetSelectedObjectsComponent2(2);

            swModel.ClearSelection2(true);

            bRet = swMoldBaseComp.Select2(false, 0);
            nRetVal = swAssy.EditPart2(true, true, ref nInfo);
            bRet = swCoreComp1.Select2(true, 0);
            swAssy.InsertCavity4(0.0, 0, 0.0, true, (int)swCavityScaleType_e.swAboutCentroid, 0);
            swAssy.EditAssembly();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Cavity Example (VBA)

This example shows how to insert a cavity in a mold base.

```
'-----
' Preconditions:
' 1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt
'    to that folder.
' 2. Click File > New > Part > OK to create a new part document.
'    a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
'    b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
'    c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
' 3. Click File > New > Assembly > OK to create a new assembly document.
'    a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
'    b. Click OK.
'    c. Click Insert Components on the Assembly toolbar, click Browse in
'       Part/Assembly to Insert, click telephone.sldprt located in the folder
'       created in step 1, and click Open.
'    d. Drop telephone.sldprt in the graphics area.
'    e. Click View > Display > Hidden Lines Visible.
'    f. In the FeatureManager design tree, click telephone<1> and click Move Component
'       in the Assembly toolbar.
'    g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
'       TIP: Change the view orientation to Top and Front to help center telephone<1>.
'    h. Click File > Save All > Rebuild and save the document (recommended),
'       navigate to the folder created in step 1, type AssemInterim.sldasm in
'       File name, and click Save.
' 4. Click telephoneMoldBase<1> in the FeatureManager design tree.
' 5. Ctrl+click telephone<1> in the FeatureManager design tree.
'
' Postconditions:
' 1. Creates Cavity1 in telephoneMoldBase<1>.
' 2. Expand telephoneMoldBase<1> to verify step 1.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swMoldBaseComp As SldWorks.Component2
    Dim swCoreComp1 As SldWorks.Component2
    Dim nRetVal As Long
    Dim nInfo As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel

    Set swSelMgr = swModel.SelectionManager
    Set swMoldBaseComp = swSelMgr.GetSelectedObjectsComponent2(1)
    Set swCoreComp1 = swSelMgr.GetSelectedObjectsComponent2(2)

    swModel.ClearSelection2 True

    bRet = swMoldBaseComp.Select2(False, 0)
    nRetVal = swAssy.EditPart2(True, True, nInfo)
    bRet = swCoreComp1.Select2(True, 0)
    swAssy.InsertCavity4 0#, 0, 0#, True, swAboutCentroid, 0
    swAssy.EditAssembly

End Sub
```

Insert Cavity Example (VB.NET)

This example shows how to insert a cavity in a mold base.

```

Preconditions:
1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt to that folder.
2. Click File > New > Part > OK to create a new part document.
   a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
   b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
   c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
3. Click File > New > Assembly > OK to create a new assembly document.
   a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
   b. Click OK.
   c. Click Insert Components on the Assembly toolbar, click Browse in Part/Assembly to Insert, click telephone.sldprt located in the folder created in step 1, and click Open.
   d. Drop telephone.sldprt in the graphics area.
   e. Click View > Display > Hidden Lines Visible.
   f. In the FeatureManager design tree, click telephone<1> and click Move Component in the Assembly toolbar.
   g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
   TIP: Change the view orientation to Top and Front to help center telephone<1>.
   h. Click File > Save All > Rebuild and save the document (recommended), navigate to the folder created in step 1, type AssemInterim.sldasm in File name, and click Save.
4. Click telephoneMoldBase<1> in the FeatureManager design tree.
5. Ctrl+click telephone<1> in the FeatureManager design tree.

Postconditions:
1. Creates Cavity1 in telephoneMoldBase<1>.
2. Expand telephoneMoldBase<1> to verify step 1.

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swAssy As AssemblyDoc
        Dim swMoldBaseComp As Component2
        Dim swCoreComp1 As Component2
        Dim nRetVal As Integer
        Dim nInfo As Integer
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swAssy = swModel

        swSelMgr = swModel.SelectionManager
        swMoldBaseComp = swSelMgr.GetSelectedObjectsComponent2(1)
        swCoreComp1 = swSelMgr.GetSelectedObjectsComponent2(2)

        swModel.ClearSelection2(True)

        bRet = swMoldBaseComp.Select2(False, 0)
        nRetVal = swAssy.EditPart2(True, True, nInfo)
        bRet = swCoreComp1.Select2(True, 0)
        swAssy.InsertCavity4(0.0#, 0, 0.0#, True, swCavityScaleType_e.swAboutCentroid, 0)
        swAssy.EditAssembly()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```
End Class
```

Insert Center of Mass Feature Example (C#)

This example shows how to insert Center of Mass and Center of Mass Reference Point features.

```
//-----  
// Preconditions: Open a part.  
//  
// Postconditions: The FeatureManager design tree contains:  
// * Center of Mass  
// * Center of Mass Reference Point1  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace InsertCenterOfMass_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 Part;  
        Feature COM;  
        Feature COMRP;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.ActiveDoc;  
  
            COM = Part.FeatureManager.InsertCenterOfMass();  
            COMRP = Part.FeatureManager.InsertCenterOfMassReferencePoint();  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Insert Center of Mass Feature Example (VBA)

This example shows how to insert Center of Mass and Center of Mass Reference Point features.

```
'-----  
' Preconditions: Open a part.  
'  
' Postconditions: The FeatureManager design tree contains:  
' * Center of Mass  
' * Center of Mass Reference Point1  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim COM As SldWorks.Feature  
Dim COMRP As SldWorks.Feature  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set COM = Part.FeatureManager.InsertCenterOfMass  
    Set COMRP = Part.FeatureManager.InsertCenterOfMassReferencePoint  
  
End Sub
```

Insert Center of Mass Feature Example (VB.NET)

This example shows how to insert Center of Mass and Center of Mass Reference Point features.

```
'-----  
' Preconditions: Open a part.  
'  
' Postconditions: The FeatureManager design tree contains:  
' * Center of Mass  
' * Center of Mass Reference Point1  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2  
    Dim COM As Feature  
    Dim COMRP As Feature
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        COM = Part.FeatureManager.InsertCenterOfMass  
        COMRP = Part.FeatureManager.InsertCenterOfMassReferencePoint
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Insert Chain Dimensions Example (C#)

This example shows how to insert chain dimensions in a drawing.

```
//=====
// Preconditions:
// 1. Open install_dir\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Select the Sheet2 tab at the bottom.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Observe the chained dimensions in Drawing View3.
// 2. Inspect the display dimension values in the Immediate window.
//
// NOTE: Because the sample is used elsewhere, do not save changes.
//=====
```

```
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace InsertChainDimensions_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDocExtension modDocExt;
            ModelDoc2 Part;
            SelectionMgr selectMgr;
            object[] dimArray = new object[4];
            object dimObject;
            object[] entityArray = new object[4];
            object varArray;
            DisplayDimension myDisplayDim;
            Dimension swDim;
            string dimText;
            int k;
            bool boolstatus;

            Part = (ModelDoc2)swApp.ActiveDoc;
            selectMgr = (SelectionMgr)Part.SelectionManager;
            modDocExt = Part.Extension;

            Part.ClearSelection2(true);
            boolstatus = ((DrawingDoc)Part).ActivateView("Drawing View3");
            boolstatus = Part.Extension.SelectByRay(0.107406727925462, 0.259964392021715, 375.00575, 0, 0, -1, 0.00193314752083778,
1, false, 0, 0);
            entityArray[0] = selectMgr.GetSelectedObject6(1, -1);
```

```

        boolstatus = Part.Extension.SelectByRay(0.135835367937783, 0.281001585630832, 375.00575, 0, 0, -1, 0.00193314752083778,
1, false, 0, 0);
        entityArray[1] = selectMgr.GetSelectedObject6(2, -1);

        boolstatus = Part.Extension.SelectByRay(0.140383950339754, 0.25598438241999, 375.00575, 0, 0, -1, 0.00193314752083778,
1, false, 0, 0);
        entityArray[2] = selectMgr.GetSelectedObject6(3, -1);

        boolstatus = Part.Extension.SelectByRay(0.176772609555524, 0.221301441604959, 375.00275, 0, 0, -1, 0.00193314752083778,
1, false, 0, 0);
        entityArray[3] = selectMgr.GetSelectedObject6(4, -1);

        varArray = entityArray;

        dimObject = modDocExt.InsertChainDimensions(varArray);

        dimArray = (object[])dimObject;

        if (dimArray.Length > 0)
        {
            for (k = 0; k <= dimArray.GetUpperBound(0); k++)
            {
                myDisplayDim = (DisplayDimension)dimArray[k];
                swDim = myDisplayDim.GetDimension2(0);
                dimText = swDim.Value.ToString();
                Debug.Print(dimText);
            }
        }
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```

Insert Chain Dimensions Example (VB.NET)

This example shows how to insert chain dimensions in a drawing.

```

=====
' Preconditions:
' 1. Open install_dir\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Select the Sheet2 tab at the bottom.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Observe the chained dimensions in Drawing View3.
' 2. Inspect the display dimension values in the Immediate window.
'
' NOTE: Because the sample is used elsewhere, do not save changes to it.
=====

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim modDocExt As ModelDocExtension
    Dim Part As ModelDoc2
    Dim selectMgr As SelectionMgr
    Dim dimArray(3) As Object
    Dim dimObject As Object
    Dim entityArray(3) As Object
    Dim varArray As Object
    Dim myDisplayDim As DisplayDimension
    Dim swDim As Dimension
    Dim dimText As String
    Dim k As Integer
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc
        selectMgr = Part.SelectionManager
        modDocExt = Part.Extension

        Part.ClearSelection2(True)
        boolstatus = Part.ActivateView("Drawing View3")
        boolstatus = Part.Extension.SelectByRay(0.107406727925462, 0.259964392021715, 375.00575, 0, 0, -1, 0.00193314752083778, 1,
False, 0, 0)
        entityArray(0) = selectMgr.GetSelectedObject6(1, -1)

        boolstatus = Part.Extension.SelectByRay(0.135835367937783, 0.281001585630832, 375.00575, 0, 0, -1, 0.00193314752083778, 1,
False, 0, 0)
        entityArray(1) = selectMgr.GetSelectedObject6(2, -1)

        boolstatus = Part.Extension.SelectByRay(0.140383950339754, 0.25598438241999, 375.00575, 0, 0, -1, 0.00193314752083778, 1,
False, 0, 0)
        entityArray(2) = selectMgr.GetSelectedObject6(3, -1)

        boolstatus = Part.Extension.SelectByRay(0.176772609555524, 0.221301441604959, 375.00275, 0, 0, -1, 0.00193314752083778, 1,
False, 0, 0)

```

```

entityArray(3) = selectMgr.GetSelectedObject6(4, -1)

varArray = entityArray

dimObject = modDocExt.InsertChainDimensions(varArray)

dimArray = dimObject

If dimArray.Length > 0 Then
    For k = 0 To UBound(dimArray)
        myDisplayDim = dimArray(k)
        swDim = myDisplayDim.GetDimension2(0)
        dimText = swDim.Value
        Debug.Print(dimText)
    Next k
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Insert Column in BOM Table Example (C#)

This example shows how to insert a part number column in a BOM table.

```
//-----
// Preconditions:
// 1. Open a drawing that contains a BOM table.
// 2. Right-click the BOM table, select Select,
//    and select Table.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a part number column at the end of the
//    the BOM table.
// 2. Examine the BOM table and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace InsertColumn2TableAnnotationCSharp.csproj
{
    partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            bool boolstatus = false;
            SelectionMgr SelMgr = default(SelectionMgr);
            TableAnnotation theTableAnnotation = default(TableAnnotation);
            int SelObjType = 0;
            int TableAnnotationType = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)swModel.SelectionManager;
            SelObjType = SelMgr.GetSelectedObjectType3(1, -1);
            if (SelObjType != (int)swSelectType_e.swSelANNOTATIONTABLES)
            {
                MessageBox.Show("Select a BOM table in the drawing before running this example.");
                return;
            }
            theTableAnnotation = (TableAnnotation)SelMgr.GetSelectedObject6(1, -1);
            TableAnnotationType = theTableAnnotation.Type;
            if (TableAnnotationType != (int)swTableAnnotationType_e.swTableAnnotation_BillofMaterials)
            {
                MessageBox.Show("Select a BOM table in the drawing before running this example.");
                return;
            }
            Debug.Print("Table before inserting a column...");
            // Display table before inserting a column
            DisplayTableColumnProps(theTableAnnotation);

            // Insert new column
            boolstatus = theTableAnnotation.InsertColumn2((int)swTableItemInsertPosition_e.swTableItemInsertPosition_Last, 0, "New
Column", (int)swInsertTableColumnWidthStyle_e.swInsertColumn_DefaultWidth);
            boolstatus = theTableAnnotation.SetColumnType2(theTableAnnotation.ColumnCount - 1,
(int)swTableColumnType_e.swBomTableColumnType_PartNumber, true);

            Debug.Print(" ");

            Debug.Print("Table after inserting a column...");
            // Display table after inserting a column
            DisplayTableColumnProps(theTableAnnotation);
        }
    }
}
```

```

public void DisplayTableColumnProps(TableAnnotation theTableAnnotation)
{
    int ColCount = 0;
    int i = 0;
    string iString = null;
    int ColType = 0;
    string ColTypeString = null;
    string ColTitle = null;
    Debug.Print("Col# " + "Type " + "Title");
    ColCount = theTableAnnotation.ColumnCount;
    for (i = 0; i <= ColCount - 1; i++)
    {
        ColType = theTableAnnotation.GetColumnType2(i, true);
        ColTypeString = System.Convert.ToString(ColType);
        ColTitle = theTableAnnotation.GetColumnTitle2(i, true);
        iString = System.Convert.ToString(i);
        Debug.Print(iString + "    " + ColTypeString + "    " + ColTitle);
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Insert Column in BOM Table Example (VBA)

This example shows how to insert a part number column in a BOM table.

```
'-----
' Preconditions:
' 1. Open a drawing that contains a BOM table.
' 2. Right-click the BOM table, select Select,
'    and select Table.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a part number column at the end of the
'    the BOM table.
' 2. Examine the BOM table and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim SelMgr As Object
Dim theTableAnnotation As SldWorks.TableAnnotation
Dim SelObjType As Long
Dim TableAnnotationType As Long

Sub DisplayTableColumnProps(theTableAnnotation As Object)

    Dim ColCount As Long
    Dim i As Long
    Dim ColType As swTableColumnTypes_e
    Dim ColTitle As String

    Debug.Print "Col#"; vbTab; "Type"; vbTab; "Title"
    ColCount = theTableAnnotation.ColumnCount
    For i = 0 To ColCount - 1
        ColType = theTableAnnotation.GetColumnType2(i, True)
        ColTitle = theTableAnnotation.GetColumnName2(i, True)
        Debug.Print i; vbTab; ColType; vbTab; ColTitle
    Next i

End Sub

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set SelMgr = swModel.SelectionManager

    SelObjType = SelMgr.GetSelectedObjectType3(1, -1)
    If SelObjType <> swSelANNOTATIONTABLES Then
        MsgBox "Select a BOM table in the drawing before running this example."
        End
    End If

    Set theTableAnnotation = SelMgr.GetSelectedObject6(1, -1)
    TableAnnotationType = theTableAnnotation.Type
    If TableAnnotationType <> swTableAnnotation_BillofMaterials Then
        MsgBox "Select a BOM table in the drawing before running this example."
        End
    End If

    Debug.Print "Table before inserting a column..."
    ' Display table before inserting a column
```

```
DisplayTableColumnProps theTableAnnotation
```

```
' Insert new column
```

```
boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Column", swInsertColumn_DefaultWidth)
```

```
boolstatus = theTableAnnotation.SetColumnType2(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_PartNumber, True)
```

```
Debug.Print " "
```

```
Debug.Print "Table after inserting a column..."
```

```
' Display table after inserting a column
```

```
DisplayTableColumnProps theTableAnnotation
```

```
End Sub
```

Insert Column in BOM Table Example (VB.NET)

This example shows how to insert a part number column in a BOM table.

```
'-----
' Preconditions:
' 1. Open a drawing that contains a BOM table.
' 2. Right-click the BOM table, select Select,
'    and select Table.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a part number column at the end of the
'    the BOM table.
' 2. Examine the BOM table and Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim boolstatus As Boolean
```

```
        Dim SelMgr As Object
```

```
        Dim theTableAnnotation As TableAnnotation
```

```
        Dim SelObjType As Long
```

```
        Dim TableAnnotationType As Long
```

```
        swModel = swApp.ActiveDoc
```

```
        SelMgr = swModel.SelectionManager
```

```
        SelObjType = SelMgr.GetSelectedObjectType3(1, -1)
```

```
        If SelObjType <> swSelectType_e.swSelANNOTATIONTABLES Then
```

```
            MsgBox("Select a BOM table in the drawing before running this example.")
```

```
            Exit Sub
```

```
        End If
```

```
        theTableAnnotation = SelMgr.GetSelectedObject6(1, -1)
```

```
        TableAnnotationType = theTableAnnotation.Type
```

```
        If TableAnnotationType <> swTableAnnotationType_e.swTableAnnotation_BillofMaterials Then
```

```
            MsgBox("Select a BOM table in the drawing before running this example.")
```

```
            Exit Sub
```

```
        End If
```

```
        Debug.Print("Table before inserting a column...")
```

```
        ' Display table before inserting a column
```

```
        DisplayTableColumnProps(theTableAnnotation)
```

```
        ' Insert new column
```

```
        boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_e.swTableItemInsertPosition_Last, 0, "New Column",
swInsertTableColumnWidthStyle_e.swInsertColumn_DefaultWidth)
```

```
        boolstatus = theTableAnnotation.SetColumnType2(theTableAnnotation.ColumnCount - 1,
swTableColumnType_e.swBomTableColumnType_PartNumber, True)
```

```
        Debug.Print(" ")
```

```
        Debug.Print("Table after inserting a column...")
```

```
        ' Display table after inserting a column
```

```
        DisplayTableColumnProps(theTableAnnotation)
```

```
    End Sub
```

```

Sub DisplayTableColumnProps(ByVal theTableAnnotation As Object)

    Dim ColCount As Long
    Dim i As Long
    Dim iString As String
    Dim ColType As Long
    Dim ColTypeString As String
    Dim ColTitle As String

    Debug.Print("Col#  " + "Type  " + "Title")

    ColCount = theTableAnnotation.ColumnCount
    For i = 0 To ColCount - 1
        ColType = theTableAnnotation.GetColumnType2(i, True)
        ColTypeString = CType(ColType, String)
        ColTitle = theTableAnnotation.GetColumnName2(i, True)
        iString = CType(i, String)
        Debug.Print(iString + "      " + ColTypeString + "      " + ColTitle)
    Next i
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Insert Conic Curve Example (C#)

This example shows how to create a conic curve.

```
//-----
// Preconditions: Verify that the specified part document template exists.
//
// Postconditions:
// 1. Creates a new part.
// 2. Inserts a conic curve in the part.
// 3. Examine the graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertConic_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;
            SketchSegment skSegment = default(SketchSegment);
            skSegment = Part.SketchManager.CreateConic(0.109436, 0.080614, 0.0, 0.048742, 0.022907, 0.0, -0.017812, 0.0, 0.0,
-0.006092,
            -0.069601, 0.0);
            Part.SketchManager.InsertSketch(true);
            Part.ShowNamedView2("Isometric", 7);
        }

        public SldWorks swApp;
    }
}
```

Insert Conic Curve Example (VBA)

This example shows how to create a conic curve.

```
'-----  
' Preconditions: Verify that the specified part document template exists.  
'  
' Postconditions:  
' 1. Creates a new part.  
' 2. Inserts a conic curve in the part.  
' 3. Examine the graphics area.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)  
    Set Part = swApp.ActiveDoc  
    Dim skSegment As SldWorks.SketchSegment  
    Set skSegment = Part.SketchManager.CreateConic(0.109436, 0.080614, 0#, 0.048742, 0.022907, 0#, -0.017812, 0#, 0#, -0.006092,  
-0.0069601, 0#)  
    Part.SketchManager.InsertSketch True  
    Part.ShowNamedView2 "Isometric", 7  
End Sub
```

Insert Conic Curve Example (VB.NET)

This example shows how to create a conic curve.

```
'-----
' Preconditions: Verify that the specified part document template exists.
'
```

```
' Postconditions:
' 1. Creates a new part.
' 2. Inserts a conic curve in the part.
' 3. Examine the graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
    Dim Part As ModelDoc2
```

```
    Sub main()

        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        Part = swApp.ActiveDoc
        Dim skSegment As SketchSegment
        skSegment = Part.SketchManager.CreateConic(0.109436, 0.080614, 0.0#, 0.048742, 0.022907, 0.0#, -0.017812, 0.0#, 0.0#,
-0.006092, -0.069601, 0.0#)
        Part.SketchManager.InsertSketch(True)
        Part.ShowNamedView2("Isometric", 7)
    End Sub
```

```
    Public swApp As SldWorks

End Class
```

Insert Connection Point Example (C#)

This example shows how to create a connection point for a tube for routing.

```
//-----
// Preconditions:
// 1. Load the SOLIDWORKS Routing Add-in (click
//   Tools > Add-Ins > SOLIDWORKS Routing).
// 2. Verify that the specified part document to open
//   exists.
//
// Postconditions:
// 1. Creates a connection point for a tube
//   using the selected edge.
// 2. Examine the graphics area and Immediate window.
//
// NOTE: Because this part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace InsertConnectionPointFeatureManagerCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatMgr = default(FeatureManager);
            int errors = 0;
            int warnings = 0;
            bool status = false;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\routing-
pipes\\fittings\\filter.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select the edge for the connection point;
            // remember to specify a value of 1 for
            // the Mark parameter for a circular edge for
            // a tube's connection point
            status = swModelDocExt.SelectByID("", "EDGE", 0.001425156111225, 0.1755840982619, -0.09117938337181, false, 1, null, 0);

            // Insert a connection point for a tube
            swFeatMgr = swModel.FeatureManager;
            Debug.Print("Connection point for tube created? " + swFeatMgr.InsertConnectionPoint((int)swConnectionPointType_e.swConnectionPoint_Tube, 0, tr
ue, 25.4 / 1000, 0.1, 0.2, 0.3, 0.4, "", 0,
0, false, "Specification", ""));
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Connection Point Example (VBA)

This example shows how to create a connection point for a tube for routing.

```
'-----
' Preconditions:
' 1. Load the SOLIDWORKS Routing Add-in (click
'   Tools > Add-Ins > SOLIDWORKS Routing).
' 2. Verify that the specified part document to open
'   exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates a connection point for a tube
'   using the selected edge.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because this part document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-
    pipes\fittings\filter.sldprt", swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Select the edge for the connection point;
    ' remember to specify a value of 1 for
    ' the Mark parameter for a circular edge for
    ' a tube's connection point
    status = swModelDocExt.SelectByID("", "EDGE", 0.001425156111225, 0.1755840982619, -0.09117938337181, False, 1, Nothing,
    0)

    ' Insert a connection point for a tube
    Set swFeatMgr = swModel.FeatureManager
    Debug.Print "Connection point for tube created? " & swFeatMgr.InsertConnectionPoint(swConnectionPoint_Tube, 0, True, 25.4
    / 1000, 0.1, 0.2, 0.3, 0.4, "", 0, 0, False, "Specification", "")
End Sub
```

Insert Connection Point Example (VB.NET)

This example shows how to create a connection point for a tube for routing.

```

-----
Preconditions:
1. Load the SOLIDWORKS Routing Add-in (click
  Tools > Add-Ins > SOLIDWORKS Routing).
2. Verify that the specified part document to open
  exists.
Postconditions:
1. Creates a connection point for a tube
  using the selected edge.
2. Examine the graphics area and Immediate window.
NOTE: Because this part document is used elsewhere,
do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatMgr As FeatureManager
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-
pipes\fittings\filter.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        ' Select the edge for the connection point;
        ' remember to specify a value of 1 for
        ' the Mark parameter for a circular edge for
        ' a tube's connection point
        status = swModelDocExt.SelectByID("", "EDGE", 0.00142515611225, 0.1755840982619, -0.09117938337181, False, 1, Nothing, 0)

        ' Insert a connection point for a tube
        swFeatMgr = swModel.FeatureManager
        Debug.Print("Connection point for tube created? " & swFeatMgr.InsertConnectionPoint(swConnectionPointType_e.swConnectionPoint_Tube, 0, True, 25.4
/ 1000, 0.1, 0.2, 0.3, 0.4, "", 0, 0, False, "Specification", ""))
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Convert Extrusion to Sheet Metal Example (C#)

This example shows how to convert a solid body to sheet metal.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\sweepcutextrude.sldprt.
//
// Postconditions:
// 1. Converts Boss-Extrude1 to sheet metal containing two rip edges.
// 2. Examine the FeatureManager design tree, which now contains:
//    * Sheet-Metal1
//    * Convert-Solid1
//    * Flat-Pattern1
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace InsertConvertSheetMetal_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swDoc = null;
            bool boolstatus = false;

            swDoc = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.008205131831119, 0.02357994168915, 0.03366815886659, true, 0,
            null, 0);
            boolstatus = swDoc.Extension.SelectByID2("", "EDGE", -0.004077318654993, 0.02376323764372, 0.04987547143355, true, 1,
            null, 0);
            boolstatus = swDoc.Extension.SelectByID2("", "EDGE", 0.02890215593544, 0.02392631827274, 0.03020230805026, true, 1,
            null, 0);
            boolstatus = swDoc.Extension.SelectByID2("", "EDGE", -0.007010951021414, 0.02376186282277, -0.0001235945334201, true,
            1, null, 0);

            // Convert extrusion to sheet metal of thickness=13mm,
            // bend radius=0.5mm, rip gap=2mm, relief type = rectangular,
            // relief ratio = 0.5, rip edge overlap type = open butt, and
            // rip edge overlap ratio = 0.5, do not keep bodies
            boolstatus = swDoc.FeatureManager.InsertConvertToSheetMetal2(0.013, false, false, 0.0005, 0.002, 0, 0.5, 1, 0.5,
            false);

            swDoc.ClearSelection2(true);
        }

        public SldWorks swApp;
    }
}
```

Convert Extrusion to Sheet Metal Example (VBA)

This example shows how to convert a solid body to sheet metal.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\sweepcutextrude.sldprt.  
' 2. Open an Immediate Window.  
'  
' Postconditions:  
' 1. Converts Boss-Extrude1 to sheet metal containing two rip edges.  
' 2. Examine the FeatureManager design tree, which now contains:  
' * Sheet-Metal1  
' * Convert-Solid1  
' * Flat-Pattern1  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim Part As SldWorks.ModelDoc2

Dim boolstatus As Boolean

Sub main()

Set swApp = Application.SldWorks

Set Part = swApp.ActiveDoc

boolstatus = Part.Extension.SelectByID("", "FACE", 4.130570195002E-04, 0.02357994168921, 0.02568415695742, True, 0, Nothing, 0)

boolstatus = Part.Extension.SelectByID("", "EDGE", -0.00190522473838, 0.02387533864419, 0.04979931166838, True, 1, Nothing, 0)

boolstatus = Part.Extension.SelectByID("", "EDGE", 0.02911271681069, 0.02376277320678, 0.02892436699148, True, 1, Nothing, 0)

boolstatus = Part.Extension.SelectByID("", "EDGE", -0.004838857104858, 0.02387396382323, -1.997542986487E-04, True, 1, Nothing, 0)

' Convert extrusion to sheet metal of thickness=13mm, bend radius=0.5mm, rip gap=2mm,

' relief type = rectangular, relief ratio = 0.5, rip edge overlap type = open butt,

' and rip edge overlap ratio = 0.5, do not keep bodies

boolstatus = Part.FeatureManager.InsertConvertToSheetMetal2(0.013, False, False, 0.0005, 0.002, 0, 0.5, 1, 0.5, false)

Part.ClearSelection2 True

End Sub

Convert Extrusion to Sheet Metal Example (VB.NET)

This example shows how to convert a solid body to sheet metal.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\sweepcutextrude.sldprt.
'
' Postconditions:
' 1. Converts Boss-Extrude1 to sheet metal containing two rip edges.
' 2. Examine the FeatureManager design tree, which now contains:
'    * Sheet-Metal1
'    * Convert-Solid1
'    * Flat-Pattern1
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Sub main()

        Dim swDoc As ModelDoc2 = Nothing
        Dim boolstatus As Boolean = False

        swDoc = CType(swApp.ActiveDoc, ModelDoc2)
        boolstatus = swDoc.Extension.SelectByID("", "FACE", -0.008205131831119, 0.02357994168915, 0.03366815886659, True, 0,
Nothing, 0)
        boolstatus = swDoc.Extension.SelectByID("", "EDGE", -0.004077318654993, 0.02376323764372, 0.04987547143355, True, 1,
Nothing, 0)
        boolstatus = swDoc.Extension.SelectByID("", "EDGE", 0.02890215593544, 0.02392631827274, 0.03020230805026, True, 1,
Nothing, 0)
        boolstatus = swDoc.Extension.SelectByID("", "EDGE", -0.007010951021414, 0.02376186282277, -0.0001235945334201, True, 1,
Nothing, 0)

        ' Convert extrusion to sheet metal of thickness=13mm, bend radius=0.5mm,
        ' rip gap=2mm, relief type = rectangular, relief ratio = 0.5,
        ' rip edge overlap type = open butt, and rip edge overlap ratio = 0.5,
        ' do not keep bodies
        boolstatus = swDoc.FeatureManager.InsertConvertToSheetMetal2(0.013, False, False, 0.0005, 0.002, 0, 0.5, 1, 0.5, false)

        swDoc.ClearSelection2(True)

    End Sub

    Public swApp As SldWorks

End Class
```

Insert Coordinate System at Center of Mass Example (VBA)

This example shows how to insert a coordinate system on the center of mass.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a 3D sketch at the center of mass.
' 2. Inserts a coordinate system at center of mass.
' 3. Examine the graphics area and the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeat As SldWorks.Feature
    Dim swModDocExt As SldWorks.ModelDocExtension
    Dim swMass As SldWorks.MassProperty
    Dim vCofG As Variant
    Dim vXaxis As Variant
    Dim vYaxis As Variant
    Dim vZaxis As Variant
    Dim swSkCofG As SldWorks.SketchPoint
    Dim swSkXaxis As SldWorks.SketchLine
    Dim swSkYaxis As SldWorks.SketchLine
    Dim swSkSegXaxis As SldWorks.SketchSegment
    Dim swSkSegYaxis As SldWorks.SketchSegment
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swModDocExt = swModel.Extension
    Set swMass = swModDocExt.CreateMassProperty
    vCofG = swMass.CenterOfMass
    vXaxis = swMass.PrincipleAxesOfInertia(0)
    vYaxis = swMass.PrincipleAxesOfInertia(1)
    vZaxis = swMass.PrincipleAxesOfInertia(2)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Mass          = " & swMass.Mass * 1000# & " g"
    Debug.Print " Surface Area    = " & swMass.SurfaceArea * 1000000# & " mm^2"
    Debug.Print " Volume          = " & swMass.Volume * 1000000000# & " mm^3"
    Debug.Print " Density          = " & swMass.Density & " kg/m^3"
    Debug.Print " CenterOfMass      = (" & vCofG(0) * 1000# & ", " & vCofG(1) * 1000# & ", " & vCofG(2) * 1000# & ") mm"
    Debug.Print " X axis            = (" & vXaxis(0) & ", " & vXaxis(1) & ", " & vXaxis(2) & ")"
    Debug.Print " Y axis            = (" & vYaxis(0) & ", " & vYaxis(1) & ", " & vYaxis(2) & ")"
    Debug.Print " Z axis            = (" & vZaxis(0) & ", " & vZaxis(1) & ", " & vZaxis(2) & ")"

    swModel.Insert3DSketch2 False
    swModel.SetAddToDB True

    Set swSkCofG = swModel.CreatePoint2(vCofG(0), vCofG(1), vCofG(2))
    Set swSkXaxis = swModel.CreateLine2(vCofG(0), vCofG(1), vCofG(2), vCofG(0) + vXaxis(0), vCofG(1) + vXaxis(1), vCofG(2) + vXaxis(2))
    Set swSkYaxis = swModel.CreateLine2(vCofG(0), vCofG(1), vCofG(2), vCofG(0) + vYaxis(0), vCofG(1) + vYaxis(1), vCofG(2) + vYaxis(2))
    Set swSkSegXaxis = swSkXaxis
    Set swSkSegYaxis = swSkYaxis

    swModel.SetAddToDB False
    swModel.Insert3DSketch2 True
    swModel.ClearSelection2 True

    swSelData.Mark = 1
    bRet = swSkCofG.Select4(True, swSelData): Debug.Assert bRet
    swSelData.Mark = 2
    bRet = swSkSegXaxis.Select4(True, swSelData): Debug.Assert bRet
    swSelData.Mark = 4
    bRet = swSkSegYaxis.Select4(True, swSelData): Debug.Assert bRet

    bRet = swModel.InsertCoordinateSystem(False, False, False): Debug.Assert bRet

End Sub
```

Insert Coordinate System Feature at Center of Mass Example (VBA)

This example shows how to insert a coordinate system feature on the center of mass.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a 3D sketch at the center of mass.
' 2. Inserts a coordinate system at center of mass.
' 3. Examine the graphics area and the Immediate window.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swModDocExt As SldWorks.ModelDocExtension
    Dim swMass As SldWorks.MassProperty
    Dim vCofG As Variant
    Dim vXaxis As Variant
    Dim vYaxis As Variant
    Dim vZaxis As Variant
    Dim swSkCofG As SldWorks.SketchPoint
    Dim swSkXaxis As SldWorks.SketchLine
    Dim swSkYaxis As SldWorks.SketchLine
    Dim swSkSegXaxis As SldWorks.SketchSegment
    Dim swSkSegYaxis As SldWorks.SketchSegment
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swFeat As SldWorks.Feature
    Dim swCoordSys As SldWorks.CoordinateSystemFeatureData
    Dim swComp As SldWorks.Component2
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swModDocExt = swModel.Extension

    Set swMass = swModDocExt.CreateMassProperty
    vCofG = swMass.CenterOfMass
    vXaxis = swMass.PrincipleAxesOfInertia(0)
    vYaxis = swMass.PrincipleAxesOfInertia(1)
    vZaxis = swMass.PrincipleAxesOfInertia(2)
    swModel.Insert3DSketch2 False
    swModel.SetAddToDB True
    Set swSkCofG = swModel.CreatePoint2(vCofG(0), vCofG(1), vCofG(2))
    Set swSkXaxis = swModel.CreateLine2(vCofG(0), vCofG(1), vCofG(2), vCofG(0) + vXaxis(0), vCofG(1) + vXaxis(1), vCofG(2) + vXaxis(2))
    Set swSkYaxis = swModel.CreateLine2(vCofG(0), vCofG(1), vCofG(2), vCofG(0) + vYaxis(0), vCofG(1) + vYaxis(1), vCofG(2) + vYaxis(2))
    Set swSkSegXaxis = swSkXaxis
    Set swSkSegYaxis = swSkYaxis
    swModel.SetAddToDB False
    swModel.Insert3DSketch2 True
    swModel.ClearSelection2 True
    swSelData.Mark = 1
    bRet = swSkCofG.Select4(True, swSelData): Debug.Assert bRet
    swSelData.Mark = 2
    bRet = swSkSegXaxis.Select4(True, swSelData): Debug.Assert bRet
    swSelData.Mark = 4
    bRet = swSkSegYaxis.Select4(True, swSelData): Debug.Assert bRet

    ' Insert the coordinate system feature, get the feature, and modify it, using the feature data
    Set swFeatMgr = swModel.FeatureManager
    Set swFeat = swFeatMgr.InsertCoordinateSystem(False, False, False)
    If swFeat Is Nothing Then
        Debug.Print "Did not get coordinate system feature."
    Else
        Set swCoordSys = swFeat.GetDefinition
        bRet = swCoordSys.AccessSelections(swModel, swComp)
        Debug.Print "XFlipped? " & swCoordSys.XFlipped
        swCoordSys.XFlipped = True

        ' Get the origin entity
        Dim gvX As Variant
        Dim vxCount As Long
        Dim i As Long

        ' Get the origin entity
        Dim entType As Long
        Dim geo As Object
        Dim oent As SldWorks.Entity
        Dim skpnt As SldWorks.SketchPoint
```

```

Set geo = swCoordSys.OriginEntity

' Is it an entity?
If TypeOf geo Is Entity Then
    Set oent = geo
    entType = geo.GetType
    Debug.Print "Entity type: " & entType
Else
    Debug.Print "Is sketch point."
    Set skpnt = geo
    If Not skpnt Is Nothing Then
        Debug.Print "Got sketch point."
    End If
End If

Dim xEnt As Object
vxCount = swCoordSys.GetXAxisEntitiesCount
gvx = swCoordSys.XAxisEntities

' Find out about the x axis entities
Dim skent As SldWorks.SketchSegment
For i = 0 To UBound(gvx)
    Set xEnt = gvx(i)
    If TypeOf xEnt Is SketchSegment Then
        Set skent = xEnt
        entType = skent.GetType
        Debug.Print "Entity type: " & entType
    End If
Next i

bRet = swFeat.ModifyDefinition(swCoordSys, swModel, Nothing)

swCoordSys.ReleaseSelectionAccess

End If
End Sub

```

Insert Cosmetic Weld Bead Example (C#)

This example shows how to insert a cosmetic weld bead feature and access an existing weld bead feature's properties.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\Insert_weld.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a fillet weld, Weld Bead2, and adds it to the Weld Folder in
// the FeatureManager design tree.
// 2. Inspect the Immediate window for Weld Bead1 settings and properties.
// 3. Expand the Weld Folder and 20.08in Fillet Weld(1) folders in the
// FeatureManager design tree to verify step 1.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            CosmeticWeldBeadFeatureData swCosmeticWeldBeadFeatureData = default(CosmeticWeldBeadFeatureData);
            CosmeticWeldBeadFolder swWeldFolder = default(CosmeticWeldBeadFolder);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            object[] edges = null;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Insert Weld Bead2
            status = swModelDocExt.SelectByID2("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("Weld Bead1", "FACE", -0.0344320907599354, 0.0170180000000641, -0.00227566098720899, false, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.InsertCosmeticWeldBead(0.51);
            swFeature = (Feature)swFeatureManager.InsertCosmeticWeldBead(0.51);

            swModel.ClearSelection2(true);

            //Get Weld Bead1 and get and set some properties
            status = swModelDocExt.SelectByID2("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, 0);
            swCosmeticWeldBeadFeatureData = (CosmeticWeldBeadFeatureData)swFeature.GetDefinition();
            swCosmeticWeldBeadFeatureData.AccessSelections(swModel, null);
            Debug.Print("Weld Bead1 settings: ");
            Debug.Print("Weld bead size: " + swCosmeticWeldBeadFeatureData.BeadSize * 39.37 + " in");
            Debug.Print("Tangent propagation? " + swCosmeticWeldBeadFeatureData.TangentPropagation);
            Debug.Print("Weld sides as defined in swCosmeticWeldBeadSide_e (1 = swCosmeticWeldBeadSide_selection): " + swCosmeticWeldBeadFeatureData.Si

de);
            Debug.Print("From/to length properties enabled? " + swCosmeticWeldBeadFeatureData.FromToLength);
            if (swCosmeticWeldBeadFeatureData.FromToLength == false)
            {
                swCosmeticWeldBeadFeatureData.FromToLength = true;
            }
            if (swCosmeticWeldBeadFeatureData.FromToLength == true)
            {
                Debug.Print("Start weld at: " + swCosmeticWeldBeadFeatureData.FromToStartPoint);
                Debug.Print("Weld length: " + swCosmeticWeldBeadFeatureData.FromToWeldLength);
                Debug.Print("Reverse weld? " + swCosmeticWeldBeadFeatureData.FromToReverse);
            }
            Debug.Print("Intermittent weld properties enabled? " + swCosmeticWeldBeadFeatureData.IntermittentWeld);
            if (swCosmeticWeldBeadFeatureData.IntermittentWeld == false)
            {
                swCosmeticWeldBeadFeatureData.IntermittentWeld = true;
            }
            if (swCosmeticWeldBeadFeatureData.IntermittentWeld == true)
            {
                if (swCosmeticWeldBeadFeatureData.GapOrPitch == true)
                {
                    Debug.Print("Intermittent gap: " + swCosmeticWeldBeadFeatureData.Gap);
                    Debug.Print("Intermittent weld length: " + swCosmeticWeldBeadFeatureData.IntermittentWeldLength);
                }
                else
                {
                    Debug.Print("Intermittent pitch: " + swCosmeticWeldBeadFeatureData.Pitch);
                    Debug.Print("Intermittent weld length: " + swCosmeticWeldBeadFeatureData.IntermittentWeldLength);
                    Debug.Print("Stagger welds if welding on both sides? " + swCosmeticWeldBeadFeatureData.Staggered);
                }
            }
            edges = (object[])swCosmeticWeldBeadFeatureData.GetReferenceEdges();
            Debug.Print("Number of reference edges: " + edges.GetUpperBound(0));

            swWeldFolder = (CosmeticWeldBeadFolder)swCosmeticWeldBeadFeatureData.GetWeldBeadFolder();

            Debug.Print("Weld Bead1 properties:");
            Debug.Print("Weld material was: " + swWeldFolder.Material);
            swWeldFolder.Material = "Steel";
            Debug.Print("Weld material is now: " + swWeldFolder.Material);
            Debug.Print("Welding cost per unit mass: " + swWeldFolder.CostPerUnitMass);
            Debug.Print("Weld mass per unit length: " + swWeldFolder.MassPerUnitLength);
            Debug.Print("Number of weld passes: " + swWeldFolder.NumberOfWeldPasses);
            Debug.Print("Weld process: " + swWeldFolder.Process);
            Debug.Print("Welding time per unit length: " + swWeldFolder.TimePerUnitLength);
            Debug.Print("Total welding cost: $" + swWeldFolder.TotalCost);
            Debug.Print("Total weld length: " + swWeldFolder.TotalLength);
        }
    }
}
```

```

        Debug.Print("    Total weld mass: " + swWeldFolder.TotalMass + " lb");
        Debug.Print("    Total number of welds: " + swWeldFolder.TotalNumber);
        Debug.Print("    Total welding time: " + swWeldFolder.TotalTime + " sec");

        status = swFeature.ModifyDefinition(swCosmeticWeldBeadFeatureData, swModel, null);
    }

    /// <summary>
    ///     The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

```

Insert Cosmetic Weld Bead Example (VBA)

This example shows how to insert a cosmetic weld bead feature and access an existing weld bead feature's properties.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\Insert_weld.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a fillet weld, Weld Bead2, and adds it to the Weld Folder in
'   the FeatureManager design tree.
' 2. Inspect the Immediate window for Weld Bead1 settings and properties.
' 3. Expand the Weld Folder and 20.08in Fillet Weld(1) folders in the
'   FeatureManager design tree to verify step 1.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swCosmeticWeldBeadFeatureData As SldWorks.CosmeticWeldBeadFeatureData
Dim swWeldFolder As SldWorks.CosmeticWeldBeadFolder
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim edges As Variant
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Insert Weld Bead2
    status = swModelDocExt.SelectByID("", "FACE", -3.44320907599354E-02, 1.701800000000641E-02, -2.27566098720899E-03, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", -1.61637176506133E-02, 5.03122973344716E-02, -1.3752238241409E-03, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.InsertCosmeticWeldBead(0.51)

    swModel.ClearSelection2 True

    'Get Weld Bead1 and get and set some properties
    status = swModelDocExt.SelectByID("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, 0)
    Set swCosmeticWeldBeadFeatureData = swFeature.GetDefinition
    swCosmeticWeldBeadFeatureData.AccessSelections swModel, Nothing

    Debug.Print "Weld Bead1 settings: "
    Debug.Print " Weld bead size: " & swCosmeticWeldBeadFeatureData.BeadSize * 39.37 & " in"
    Debug.Print " Tangent propagation? " & swCosmeticWeldBeadFeatureData.TangentPropagation
    Debug.Print " Weld sides as defined in swCosmeticWeldBeadSide_e (1 = swCosmeticWeldBeadSide_selection): " & swCosmeticWeldBeadFeatureData.Side
    Debug.Print " From/to length properties enabled? " & swCosmeticWeldBeadFeatureData.FromToLength
    If swCosmeticWeldBeadFeatureData.FromToLength = False Then
        swCosmeticWeldBeadFeatureData.FromToLength = True
    End If
    If swCosmeticWeldBeadFeatureData.FromToLength = True Then
        Debug.Print " Start weld at: " & swCosmeticWeldBeadFeatureData.FromToStartPoint
        Debug.Print " Weld length: " & swCosmeticWeldBeadFeatureData.FromToWeldLength
        Debug.Print " Reverse weld? " & swCosmeticWeldBeadFeatureData.FromToReverse
    End If
    Debug.Print " Intermittent weld properties enabled? " & swCosmeticWeldBeadFeatureData.IntermittentWeld
    If swCosmeticWeldBeadFeatureData.IntermittentWeld = False Then
        swCosmeticWeldBeadFeatureData.IntermittentWeld = True
    End If
    If swCosmeticWeldBeadFeatureData.IntermittentWeld = True Then
        If swCosmeticWeldBeadFeatureData.GapOrPitch = True Then
            Debug.Print " Intermittent gap: " & swCosmeticWeldBeadFeatureData.Gap
            Debug.Print " Intermittent weld length: " & swCosmeticWeldBeadFeatureData.IntermittentWeldLength
        Else
            Debug.Print " Intermittent pitch: " & swCosmeticWeldBeadFeatureData.Pitch
            Debug.Print " Intermittent weld length: " & swCosmeticWeldBeadFeatureData.IntermittentWeldLength
            Debug.Print " Stagger welds if welding on both sides? " & swCosmeticWeldBeadFeatureData.Staggered
        End If
    End If
    edges = swCosmeticWeldBeadFeatureData.GetReferenceEdges
    Debug.Print " Number of reference edges: " & UBound(edges)

    Set swWeldFolder = swCosmeticWeldBeadFeatureData.GetWeldBeadFolder

    Debug.Print "Weld Bead1 properties:"
    Debug.Print " Weld material was: " & swWeldFolder.Material
    swWeldFolder.Material = "Steel"
    Debug.Print " Weld material is now: " & swWeldFolder.Material
    Debug.Print " Welding cost per unit mass: " & swWeldFolder.CostPerUnitMass
    Debug.Print " Weld mass per unit length: " & swWeldFolder.MassPerUnitLength
    Debug.Print " Number of weld passes: " & swWeldFolder.NumberOfWeldPasses
    Debug.Print " Weld process: " & swWeldFolder.Process
    Debug.Print " Welding time per unit length: " & swWeldFolder.TimePerUnitLength
    Debug.Print " Total welding cost: $" & swWeldFolder.TotalCost
    Debug.Print " Total weld length: " & swWeldFolder.TotalLength
    Debug.Print " Total weld mass: " & swWeldFolder.TotalMass & " lb"

```

```
Debug.Print "    Total number of welds: " & swWeldFolder.TotalNumber
Debug.Print "    Total welding time: " & swWeldFolder.TotalTime & " sec"

status = swFeature.ModifyDefinition(swCosmeticWeldBeadFeatureData, swModel, Nothing)

End Sub
```

Insert Cosmetic Weld Bead Example (VB.NET)

This example shows how to insert a cosmetic weld bead feature and access an existing cosmetic weld bead feature's properties.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\Insert_weld.sldprt.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Inserts a fillet weld, Weld Bead2, and adds it to the Weld Folder in
'   the FeatureManager design tree.
' 2. Inspect the Immediate window for Weld Bead1 settings and properties.
' 3. Expand the Weld Folder and 20.08in Fillet Weld(1) folders in the
'   FeatureManager design tree to verify step 1.
'-----
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swCosmeticWeldBeadFeatureData As CosmeticWeldBeadFeatureData
        Dim swWeldFolder As CosmeticWeldBeadFolder
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim edges() As Object
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Insert Weld Bead2
        status = swModelDocExt.SelectByID("", "FACE", -0.0344320907599354, 0.0170180000000641, -0.00227566098720899, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "FACE", -0.0161637176506133, 0.0503122973344716, -0.0013752238241409, True, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.InsertCosmeticWeldBead(0.51)

        swModel.ClearSelection2(True)

        'Get Weld Bead1 and get and set some properties
        status = swModelDocExt.SelectByID("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, 0)
        swCosmeticWeldBeadFeatureData = swFeature.GetDefinition
        swCosmeticWeldBeadFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Weld Bead1 settings: ")
        Debug.Print("  Weld bead size: " & swCosmeticWeldBeadFeatureData.BeadSize * 39.37 & " in")
        Debug.Print("  Tangent propagation? " & swCosmeticWeldBeadFeatureData.TangentPropagation)
        Debug.Print("  Weld sides as defined in swCosmeticWeldBeadSide_e (1 = swCosmeticWeldBeadSide_selection): " & swCosmeticWeldBeadFeatureData.Side)
        Debug.Print("  From/to length properties enabled? " & swCosmeticWeldBeadFeatureData.FromToLength)
        If swCosmeticWeldBeadFeatureData.FromToLength = False Then
            swCosmeticWeldBeadFeatureData.FromToLength = True
        End If
        If swCosmeticWeldBeadFeatureData.FromToLength = True Then
            Debug.Print("  Start weld at: " & swCosmeticWeldBeadFeatureData.FromToStartPoint)
            Debug.Print("  Weld length: " & swCosmeticWeldBeadFeatureData.FromToWeldLength)
            Debug.Print("  Reverse weld? " & swCosmeticWeldBeadFeatureData.FromToReverse)
        End If
        Debug.Print("  Intermittent weld properties enabled? " & swCosmeticWeldBeadFeatureData.IntermittentWeld)
        If swCosmeticWeldBeadFeatureData.IntermittentWeld = False Then
            swCosmeticWeldBeadFeatureData.IntermittentWeld = True
        End If
        If swCosmeticWeldBeadFeatureData.IntermittentWeld = True Then
            If swCosmeticWeldBeadFeatureData.GapOrPitch = True Then
                Debug.Print("  Intermittent gap: " & swCosmeticWeldBeadFeatureData.Gap)
                Debug.Print("  Intermittent weld length: " & swCosmeticWeldBeadFeatureData.IntermittentWeldLength)
            Else
                Debug.Print("  Intermittent pitch: " & swCosmeticWeldBeadFeatureData.Pitch)
                Debug.Print("  Intermittent weld length: " & swCosmeticWeldBeadFeatureData.IntermittentWeldLength)
                Debug.Print("  Stagger welds if welding on both sides? " & swCosmeticWeldBeadFeatureData.Staggered)
            End If
        End If
        edges = swCosmeticWeldBeadFeatureData.GetReferenceEdges
        Debug.Print("  Number of reference edges: " & UBound(edges))

        swWeldFolder = swCosmeticWeldBeadFeatureData.GetWeldBeadFolder

        Debug.Print("Weld Bead1 properties:")
        Debug.Print("  Weld material was: " & swWeldFolder.Material)
        swWeldFolder.Material = "Steel"
        Debug.Print("  Weld material is now: " & swWeldFolder.Material)
        Debug.Print("  Welding cost per unit mass: " & swWeldFolder.CostPerUnitMass)
        Debug.Print("  Weld mass per unit length: " & swWeldFolder.MassPerUnitLength)
        Debug.Print("  Number of weld passes: " & swWeldFolder.NumberOfWeldPasses)
        Debug.Print("  Weld process: " & swWeldFolder.Process)
        Debug.Print("  Welding time per unit length: " & swWeldFolder.TimePerUnitLength)
        Debug.Print("  Total welding cost: $" & swWeldFolder.TotalCost)
        Debug.Print("  Total weld length: " & swWeldFolder.TotalLength)
        Debug.Print("  Total weld mass: " & swWeldFolder.TotalMass & " lb")
        Debug.Print("  Total number of welds: " & swWeldFolder.TotalNumber)
        Debug.Print("  Total welding time: " & swWeldFolder.TotalTime & " sec")

        status = swFeature.ModifyDefinition(swCosmeticWeldBeadFeatureData, swModel, Nothing)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Insert Cosmetic Weld Bead Using Geometric Entities Example (C#)

This example shows how to insert a cosmetic weld bead using geometric entities.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a cosmetic weld bead using the
//    selected geometric entities (i.e., faces).
// 2. To verify, examine the graphics area and
//    expand the Weld Folder and its subfolder
//    in the FeatureManager design tree.
// 3. Examine the Immediate window.
//
// NOTE: Because the part document is used elsewhere,
// do not save any changes to it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace InsertCosmeticWeldBead2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            SelectionMgr swSelMgr = default(SelectionMgr);
            FeatureManager swFeatureMgr = default(FeatureManager);
            CosmeticWeldBeadFeatureData swCosmeticWeldBeadFeatureData = default(CosmeticWeldBeadFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int entityType = 0;

            //Open model document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\driveworksxpess\\leg.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select the faces
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            //From face
            status = swModelDocExt.SelectByID2("", "FACE", 0.447611268878973, 0.185506718400291, 0.00676112086262037, true, 4, null, 0);
            //To face
            status = swModelDocExt.SelectByID2("", "FACE", 0.567647499089958, 0.0888999999998532, 0.00208831790428121, true, 8, null, 0);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            Face2[] weldFromFace = new Face2[1];
            object[] weldFromArray = new object[1];
            weldFromFace[0] = (Face2)swSelMgr.GetSelectedObject6(1, 4);
            weldFromArray = (object[])weldFromFace;

            Face2[] weldToFace = new Face2[1];
            object[] weldToArray = new object[1];
            weldToFace[0] = (Face2)swSelMgr.GetSelectedObject6(1, 8);
            weldToArray = (object[])weldToFace;

            //Create cosmetic weld bead using the selected faces
            object[] weldObj = new object[2];
            swFeatureMgr = swModel.FeatureManager;
            weldObj = (object[])swFeatureMgr.InsertCosmeticWeldBead2(0, weldFromArray, weldToArray, 15 / 1000);

            //Get the weld-from and weld-to entities and their types
            status = swModelDocExt.SelectByID2("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swCosmeticWeldBeadFeatureData = (CosmeticWeldBeadFeatureData)swFeature.GetDefinition();
            swCosmeticWeldBeadFeatureData.AccessSelections(swModel, null);
            Debug.Print(swFeature.Name);
            weldObj = (object[])swCosmeticWeldBeadFeatureData.GetEntitiesWeldFrom(out entityType);
            Debug.Print(" Weld-from type: " + entityType);
            weldObj = (object[])swCosmeticWeldBeadFeatureData.GetEntitiesWeldTo(out entityType);
            Debug.Print(" Weld-to type: " + entityType);
            swCosmeticWeldBeadFeatureData.ReleaseSelectionAccess();
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Insert Cosmetic Weld Bead Using Geometric Entities Example (VBA)

This example shows how to insert a cosmetic weld bead using geometric entities.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a cosmetic weld bead using the
'    selected geometric entities (i.e., faces).
' 2. To verify, examine the graphics area and
'    expand the Weld Folder and its subfolder
'    in the FeatureManager design tree.
' 3. Examine the Immediate window.
'
' NOTE: Because this part document is used elsewhere,
' do not save any changes to it.
'-----
Option Explicit

Dim SwApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swSelMgr As SldWorks.SelectionMgr
Dim swCosmeticWeldBeadFeatureData As SldWorks.CosmeticWeldBeadFeatureData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim entityType As Long

Sub main()

    Set SwApp = Application.SldWorks
    'Open model document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\driveworks\express\leg.sldprt"
    Set swModel = SwApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select the faces
    Set swModelDocExt = swModel.Extension
    'From face
    status = swModelDocExt.SelectByID("", "FACE", 0.447611268878973, 0.185506718400291, 6.76112086262037E-03, True, 4, Nothing, 0)
    'To face
    status = swModelDocExt.SelectByID("", "FACE", 0.567647499089958, 8.88999999998532E-02, 2.08831790428121E-03, True, 8, Nothing, 0)

    Set swSelMgr = swModel.SelectionManager
    Dim weldFromFace(0) As SldWorks.Face2
    Dim weldFromArray As Variant
    Set weldFromFace(0) = swSelMgr.GetSelectedObject6(1, 4)
    weldFromArray = weldFromFace

    Dim weldToFace(0) As SldWorks.Face2
    Dim weldToArray As Variant
    Set weldToFace(0) = swSelMgr.GetSelectedObject6(1, 8)
    weldToArray = weldToFace

    'Create cosmetic weld bead using the selected faces
    Dim weldObjs As Variant
    Set swFeatureMgr = swModel.FeatureManager
    weldObjs = swFeatureMgr.InsertCosmeticWeldBead2(0, weldFromArray, weldToArray, 15 / 1000)

    'Get the weld-from and weld-to entities and their types
    status = swModelDocExt.SelectByID("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swCosmeticWeldBeadFeatureData = swFeature.GetDefinition
    swCosmeticWeldBeadFeatureData.AccessSelections swModel, Nothing
    Debug.Print swFeature.Name
    weldObjs = swCosmeticWeldBeadFeatureData.GetEntitiesWeldFrom(entityType)
    Debug.Print " Weld-from type: " & entityType
    weldObjs = swCosmeticWeldBeadFeatureData.GetEntitiesWeldTo(entityType)
    Debug.Print " Weld-to type: " & entityType
    swCosmeticWeldBeadFeatureData.ReleaseSelectionAccess

    swModel.ClearSelection2 True

End Sub
```

Insert Cosmetic Weld Bead Using Geometric Entities Example (VB.NET)

This example shows how to insert a cosmetic weld bead using geometric entities.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a cosmetic weld bead using the
'    selected geometric entities (i.e., faces).
' 2. To verify, examine the graphics area and
'    expand the Weld Folder and its subfolder
'    in the FeatureManager design tree.
' 3. Examine the Immediate window.
'
' NOTE: Because the part document is used elsewhere,
' do not save any changes to it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swSelMgr As SelectionMgr
        Dim swFeatureMgr As FeatureManager
        Dim swCosmeticWeldBeadFeatureData As CosmeticWeldBeadFeatureData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim entityType As Integer

        'Open model document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\driveworks\express\leg.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select the faces
        swModelDocExt = swModel.Extension
        'From face
        status = swModelDocExt.SelectByID("", "FACE", 0.447611268878973, 0.185506718400291, 0.00676112086262037, True, 4, Nothing, 0)
        'To face
        status = swModelDocExt.SelectByID("", "FACE", 0.567647499089958, 0.08889999999998532, 0.00208831790428121, True, 8, Nothing, 0)

        swSelMgr = swModel.SelectionManager
        Dim weldFromFace(0) As Face2
        Dim weldFromArray(0) As Object
        weldFromFace(0) = swSelMgr.GetSelectedObject6(1, 4)
        weldFromArray = weldFromFace

        Dim weldToFace(0) As Face2
        Dim weldToArray(0) As Object
        weldToFace(0) = swSelMgr.GetSelectedObject6(1, 8)
        weldToArray = weldToFace

        'Create cosmetic weld bead using the selected faces
        Dim weldObjs(1) As Object
        swFeatureMgr = swModel.FeatureManager
        weldObjs = swFeatureMgr.InsertCosmeticWeldBead2(0, weldFromArray, weldToArray, 15 / 1000)

        'Get the weld-from and weld-to entities and their types
        status = swModelDocExt.SelectByID("Weld Bead1", "COSMETIC_WELD", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swCosmeticWeldBeadFeatureData = swFeature.GetDefinition
        swCosmeticWeldBeadFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print(swFeature.Name)
        weldObjs = swCosmeticWeldBeadFeatureData.GetEntitiesWeldFrom(entityType)
        Debug.Print(" Weld-from type: " & entityType)
        weldObjs = swCosmeticWeldBeadFeatureData.GetEntitiesWeldTo(entityType)
        Debug.Print(" Weld-to type: " & entityType)
        swCosmeticWeldBeadFeatureData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert Cut Extrude Example (C#)

This example shows how to insert a cut extrude feature.

```
//-----
// Preconditions: Verify that specified part to open exists.
//
// Postconditions:
// 1. Inserts a cut extrude feature is in the model.
// 2. Examine the graphics area.
//
// NOTE: Because the part document is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace FeatureCut3FeatureManagerCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool boolstatus = false;
            int fileerror = 0;
            int filewarning = 0;

            // Open part document
            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\plate.sldprt",
(int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref fileerror, ref filewarning);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select the face where to sketch a circle
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02031412853728, 0.006977746007294, -0.008053400767039, false, 0,
null, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            // Sketch a circle
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.01708, -0.030458, 0.0);
            boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);

            // Insert a cut-extrude feature using the circle
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureCut3(true, false, false, (int)swEndConditions_e.swEndCondThroughAll,
(int)swEndConditions_e.swEndCondBlind, 0.01, 0.01, false, false, false,
false, 0.01745329251994, 0.01745329251994, false, false, false, false, true, true,
false, false, false, (int)swStartConditions_e.swStartSketchPlane, 0, false);
        }

        /// <summary>
        /// The Sldworks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Cut Extrude Example (VBA)

This example shows how to insert a cut extrude feature.

```
'-----
' Preconditions: Verify that the specified file to open exists.
'
' Postconditions:
' 1. Inserts a cut extrude feature in the model.
' 2. Examine the graphics area.
'
' NOTE: Because the part document is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim boolstatus As Boolean
Dim fileerror As Long, filewarning As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open document
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\plate.sldprt", swDocPART, swOpenDocOptions_Silent, "",
fileerror, filewarning
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Select the face where to sketch a circle
    boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02031412853728, 0.006977746007294, -0.008053400767039, False, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    ' Sketch a circle
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.01708, -0.030458, 0#)
    boolstatus = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True

    ' Create a cut-extrude feature using the circle
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureCut3(True, False, False, swEndCondThroughAll, swEndCondBlind, 0.01, 0.01, False, False, False, False,
0.01745329251994, 0.01745329251994, False, False, False, False, False, True, True, False, False, False, swStartSketchPlane, 0, False)

End Sub
```

Insert Cut Extrude Example (VB.NET)

This example shows how to insert a cut extrude feature.

```
'-----
' Preconditions: Verify that the specified file to open exists.
'
' Postconditions:
' 1. Inserts a cut extrude feature in the model.
' 2. Examine the graphics area.
'
' NOTE: Because the part document is used elsewhere, do not save
' changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
```

Partial Class SolidWorksMacro

```
    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim boolstatus As Boolean
        Dim fileerror As Long, filewarning As Long

        ' Open part document
        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\plate.sldprt",
        swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", fileerror, filewarning)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Select the face where to sketch a circle
        boolstatus = swModelDocExt.SelectByID("", "FACE", -0.02031412853728, 0.006977746007294, -0.008053400767039, False, 0,
        Nothing, 0)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        ' Sketch a circle
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.01708, -0.030458, 0.0#)
        boolstatus = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        ' Create a cut-extrude feature using the circle
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureCut3(True, False, False, swEndConditions_e.swEndCondThroughAll,
        swEndConditions_e.swEndCondBlind, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False,
        False, False, True, True, False, False, False, swStartConditions_e.swStartSketchPlane, 0, False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Insert Cut List Item Property Note Example (C#)

This example shows how to insert a note containing all of the cut list item properties into the selected flat pattern drawing view of a sheet metal part.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\2012-sm.slddrw.
//
// Postconditions:
// 1. Inserts a note with all of the cut list item properties in Drawing View1.
// 2. Examine the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertCutListPropertyNote_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr SelMgr;
        View aView;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            SelMgr = (SelectionMgr)Part.SelectionManager;
            aView = (View)SelMgr.GetSelectedObject6(1, -1);
            aView.InsertCutListPropertyNote(0.11, 0.17, 0.0);
        }

        public SldWorks swApp;
    }
}
```

Insert Cut List Item Property Note Example (VBA)

This example shows how to insert a note containing all of the cut list item properties into the selected flat pattern drawing view of a sheet metal part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\2012-sm.slddrw.  
'  
' Postconditions:  
' 1. Inserts a note with all of the cut list item properties in Drawing View1.  
' 2. Examine the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Option Explicit
```

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim SelMgr As SldWorks.SelectionMgr  
Dim aView As SldWorks.View  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.Extension.SelectByID2("Drawing  
View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
    Set SelMgr = Part.SelectionManager  
    Set aView = SelMgr.GetSelectedObject6(1, -1)  
    aView.InsertCutListPropertyNote 0.11, 0.17, 0#  
  
End Sub
```

Insert Cut List Item Property Note Example (VB.NET)

This example shows how to insert a note containing all of the cut list item properties into the selected flat pattern drawing view of a sheet metal part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\2012-sm.slddrw.  
'  
' Postconditions:  
' 1. Inserts a note with all of the cut list item properties in Drawing View1.  
' 2. Examine the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim SelMgr As SelectionMgr
```

```
    Dim aView As View
```

```
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID2("Drawing  
View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
```

```
        SelMgr = Part.SelectionManager
```

```
        aView = SelMgr.GetSelectedObject6(1, -1)
```

```
        aView.InsertCutListPropertyNote(0.11, 0.17, 0.0#)
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Insert Body-Delete/Keep Feature Example (C#)

This example shows how to insert a Body-Delete/Keep feature into a multibody part.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates Body-Delete/Keep 1 in the FeatureManager design tree.
// 2. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertDeleteBody_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        DeleteBodyFeatureData insDelBody;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            // Select body to delete
            boolstatus = Part.Extension.SelectByID2("Fillet5", "SOLIDBODY", 0.0592851881957586, 0.0409115950836281,
            -0.0197275812591329, true, 0, null, 0);
            // Create a Body-Delete/Keep feature
            myFeature = Part.FeatureManager.InsertDeleteBody2(false);

            insDelBody = (DeleteBodyFeatureData)myFeature.GetDefinition();
            Debug.Print("Number of bodies in this Body-Delete/Keep feature: " + insDelBody.GetBodiesCount());
            Debug.Print("Keep bodies: " + insDelBody.KeepBodies);
        }

        public SldWorks swApp;
    }
}
```

Insert Body-Delete/Keep Feature Example (VBA)

This example shows how to insert a Body-Delete/Keep feature into a multibody part.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates Body-Delete/Keep 1 in the FeatureManager design tree.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myFeature As SldWorks.Feature
Dim insDelBody As SldWorks.DeleteBodyFeatureData
Dim boolstatus As Boolean
Option Explicit

Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    ' Select body to delete
    boolstatus = Part.Extension.SelectByID2("Fillet5", "SOLIDBODY", 5.92851881957586E-02, 4.09115950836281E-02, -1.97275812591329E-02, True, 0, Nothing, 0)
    ' Create a Body-Delete/Keep feature
    Set myFeature = Part.FeatureManager.InsertDeleteBody2(False)

    Set insDelBody = myFeature.GetDefinition
    Debug.Print "Number of bodies in this Body-Delete/Keep feature: " & insDelBody.GetBodiesCount
    Debug.Print "Keep bodies: " & insDelBody.KeepBodies
End Sub
```

Insert Body-Delete/Keep Feature Example (VB.NET)

This example shows how to insert a Body-Delete/Keep feature into a multibody part.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates Body-Delete/Keep 1 in the FeatureManager design tree.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim myFeature As Feature
    Dim insDelBody As DeleteBodyFeatureData
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc
        ' Select body to delete
        boolstatus = Part.Extension.SelectByID2("Fillet5", "SOLIDBODY", 0.0592851881957586, 0.0409115950836281,
-0.0197275812591329, True, 0, Nothing, 0)
        ' Create a Body-Delete/Keep feature
        myFeature = Part.FeatureManager.InsertDeleteBody2(False)

        insDelBody = myFeature.GetDefinition
        Debug.Print("Number of bodies in this Body-Delete/Keep feature: " & insDelBody.GetBodiesCount)
        Debug.Print("Keep bodies: " & insDelBody.KeepBodies)

    End Sub

    Public swApp As SldWorks

End Class

```

Insert DXF/DWG File and Add Dimensions Example (VBA)

This example shows how to insert a DXF/DWG image on a preselected plane or face and then autodimension it.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Replace path_name with the pathname of an existing DXF/DWG file.
' 3. Select a plane or face on which to insert the DXF/DWG image.
'
' Postconditions:
' 1. Adds the DXF/DWG image as a sketch.
' 2. Autodimensions the sketch.
' 3. Use Zoom to Area to inspect the sketch dimensioning.
' 4. Press F5 to rebuild the model.
'-----
Option Explicit

Const nTolerance As Double = 0.00000001
Dim swModelView As SldWorks.ModelView
Dim swSketchMgr As SldWorks.SketchManager
Dim nRetVal As Long

Sub main()
    Const sDwgFileName As String = "path_name"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim bRet As Boolean
    Dim importData As SldWorks.ImportDxfDwgData

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelView = swModel.ActiveView
    Set swFeatMgr = swModel.FeatureManager
    Set importData = swApp.GetImportFileData(sDwgFileName)

    'Unit
    importData.LengthUnit("") = SwConst.swLengthUnit_e.swMM

    'Position
    bRet = importData.SetPosition("", swDwgEntitiesCentered, 0, 0)

    'Sheet scale
    bRet = importData.SetSheetScale("", 1#, 2#)

    'Paper size
    bRet = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperASize, 0#, 0#)

    'Import method
    importData.ImportMethod("") = swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingPart
    Set swFeat = swFeatMgr.InsertDwgOrDxfFile2(sDwgFileName, importData)

    Set swSketch = swFeat.GetSpecificFeature2
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    nRetVal = AutoDimensionSketch(swApp, swModel, swSketch, swSelData)

    Stop

    ' Rebuild to update sketch
    swModel.EditRebuild3
End Sub
```

Function GetAllSketchLines(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch) As Variant

```
Dim vSketchSegArr As Variant
Dim vSketchSeg As Variant
Dim swSketchSeg As SldWorks.SketchSegment
Dim swSketchCurrLine As SldWorks.SketchLine
Dim swSketchLineArr( ) As SldWorks.SketchLine
ReDim swSketchLineArr(0)

vSketchSegArr = swSketch.GetSketchSegments

If Not IsEmpty(vSketchSegArr) Then
    For Each vSketchSeg In vSketchSegArr
        Set swSketchSeg = vSketchSeg
        If swSketchLINE = swSketchSeg.GetType Then
            Set swSketchCurrLine = swSketchSeg
            Set swSketchLineArr(UBound(swSketchLineArr)) = swSketchCurrLine
            ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) + 1)
        End If
    Next
End If
```

```

        Next
    End If
    If 0 = UBound(swSketchLineArr) Then
        ' No straight lines in this sketch
        GetAllSketchLines = Empty
        Exit Function
    End If
    ' Remove last, empty sketch line
    ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) - 1)
    GetAllSketchLines = swSketchLineArr
End Function

Function GetSketchPoint(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchPt As SldWorks.SketchPoint) As Boolean

    Dim vSketchPtArr As Variant
    vSketchPtArr = swSketch.GetSketchPoints2
    If Not IsEmpty(vSketchPtArr) Then
        ' Use first point
        Set swSketchPt = vSketchPtArr(0)
        GetSketchPoint = True
        Exit Function
    End If
    GetSketchPoint = False
End Function

Function FindVerticalOrigin(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchSegVert As SldWorks.SketchSegment, swSketchPtVert As SldWorks.SketchPoint) As Boolean

    Dim vSketchLineArr As Variant
    Dim vSketchLine As Variant
    Dim swSketchCurrLine As SldWorks.SketchLine
    Dim swStartPt As SldWorks.SketchPoint
    Dim swEndPt As SldWorks.SketchPoint

    ' Get first vertical line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)
    If Not IsEmpty(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            Set swSketchCurrLine = vSketchLine
            Set swStartPt = swSketchCurrLine.GetStartPoint2
            Set swEndPt = swSketchCurrLine.GetEndPoint2

            If Abs(swStartPt.X - swEndPt.X) < nTolerance Then
                Set swSketchSegVert = swSketchCurrLine
                FindVerticalOrigin = True
                Exit Function
            End If
        Next
    End If

    ' Get first point
    FindVerticalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert)
End Function

Function FindHorizontalOrigin(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchSegHoriz As SldWorks.SketchSegment, swSketchPtHoriz As SldWorks.SketchPoint) As Boolean

    Dim vSketchLineArr As Variant
    Dim vSketchLine As Variant
    Dim swSketchCurrLine As SldWorks.SketchLine
    Dim swStartPt As SldWorks.SketchPoint
    Dim swEndPt As SldWorks.SketchPoint

    ' Get first horizontal line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)
    If Not IsEmpty(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            Set swSketchCurrLine = vSketchLine
            Set swStartPt = swSketchCurrLine.GetStartPoint2
            Set swEndPt = swSketchCurrLine.GetEndPoint2

            If Abs(swStartPt.Y - swEndPt.Y) < nTolerance Then
                Set swSketchSegHoriz = swSketchCurrLine
                FindHorizontalOrigin = True
                Exit Function
            End If
        Next
    End If

    ' Get first point
    FindHorizontalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz)
End Function

Function AutoDimensionSketch(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSelData As SldWorks.SelectData) As Long

    Dim swFeat As SldWorks.Feature
    Dim swSketchSegHoriz As SldWorks.SketchSegment
    Dim swSketchPtHoriz As SldWorks.SketchPoint
    Dim swSketchSegVert As SldWorks.SketchSegment
    Dim swSketchPtVert As SldWorks.SketchPoint
    Dim bRet As Boolean

    If False = FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz) Then
        AutoDimensionSketch = swAutodimStatusDatumLineNotHorizontal
        Exit Function
    End If

```

```

If False = FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert) Then
    AutoDimensionSketch = swAutodimStatusDatumLineNotVertical
    Exit Function
End If

Set swFeat = swSketch
bRet = swFeat.Select2(False, 0)

' Editing sketch clears selections
swModel.EditSketch

' Reselect sketch segments with correct marks for auto-dimensioning
If Not swSketchSegVert Is Nothing Then
    ' Vertical line is for horizontal datum
    bRet = swSketchSegVert.Select4(True, swSelData)
ElseIf Not swSketchPtHoriz Is Nothing Then
    bRet = swSketchPtHoriz.Select4(True, swSelData)
ElseIf Not swSketchPtVert Is Nothing Then
    ' Use any sketch point for horizontal datum
    bRet = swSketchPtVert.Select4(True, swSelData)
End If

If Not swSketchSegHoriz Is Nothing Then
    ' Horizontal line is for vertical datum
    bRet = swSketchSegHoriz.Select4(True, swSelData)
ElseIf Not swSketchPtVert Is Nothing Then
    bRet = swSketchPtVert.Select4(True, swSelData)
ElseIf Not swSketchPtHoriz Is Nothing Then
    ' Use any sketch point for vertical datum
    bRet = swSketchPtHoriz.Select4(True, swSelData)
End If

' No straight lines, probably contains circles
' so use sketch points for datums
If IsEmpty(GetAllSketchLines(swApp, swModel, swSketch)) Then
    If Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(False, swSelData)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(False, swSelData)
    End If
End If

Set swSketchMgr = swModel.SketchManager
nRetVal = swSketchMgr.FullyDefineSketch(True, True, swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical
Or swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, True, 1, Nothing, 1, Nothing, 1, 1)

' Redraw so dimensions are displayed immediately
Dim rect() As Double
swModelView.GraphicsRedraw (rect)

End Function

```

Insert DXF/DWG File and Add Dimensions Example (C#)

This example shows how to insert a DXF/DWG image on a preselected plane or face and then autodimension it.

```
//-----
// Preconditions:
// 1. Open a part.
// 2. Replace path_name with the pathname of an existing DXF/DWG file.
// 3. Select a plane or face on which to insert the DXF/DWG image.
//
// Postconditions:
// 1. Adds the DXF/DWG image as a sketch.
// 2. Autodimensions the sketch.
// 3. Use Zoom to Area to inspect the sketch dimensioning.
// 4. Press F5 to rebuild the model.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace InsertDXFAndAutodimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        const double nTolerance = 1E-08;
        ModelView swModelView;
        SketchManager swSketchMgr;
        int nRetVal;

        public void Main()
        {
            const string sDwgFileName = "path_name";

            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            Sketch swSketch = default(Sketch);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            bool bRet = false;
            ImportDxfDwgData importData = default(ImportDxfDwgData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelView = (ModelView)swModel.ActiveView;
            swFeatMgr = swModel.FeatureManager;
            importData = (ImportDxfDwgData)swApp.GetImportFileData(sDwgFileName);

            //Unit
            importData.set_LengthUnit("", (int)swLengthUnit_e.swMM);

            //Position
            bRet = importData.SetPosition("", (int)swDwgImportEntitiesPositioning_e.swDwgEntitiesCentered, 0, 0);

            //Sheet scale
            bRet = importData.SetSheetScale("", 1.0, 2.0);

            //Paper size
            bRet = importData.SetPaperSize("", (int)swDwgPaperSizes_e.swDwgPaperAsize, 0.0, 0.0);

            //Import method
            importData.set_ImportMethod("", (int)swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingPart);

            swFeat = swFeatMgr.InsertDwgOrDxfFile2(sDwgFileName, importData);

            swSketch = (Sketch)swFeat.GetSpecificFeature2();
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();

            nRetVal = AutoDimensionSketch(swApp, swModel, swSketch, swSelData);

            System.Diagnostics.Debugger.Break();

            // Rebuild to update sketch
            swModel.EditRebuild3();
        }

        public object GetAllSketchLines(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch)
        {
            object functionReturnValue = null;

            Object[] vSketchSegArr = null;
        }
    }
}
```

```

object vSketchSeg = null;
SketchSegment swSketchSeg = default(SketchSegment);
SketchLine swSketchCurrLine = default(SketchLine);
SketchLine[] swSketchLineArr = new SketchLine[1];

vSketchSegArr = (Object[])swSketch.GetSketchSegments();

if ((vSketchSegArr != null))
{
    foreach (SketchSegment vSketchSeg_loopVariable in vSketchSegArr)
    {
        vSketchSeg = vSketchSeg_loopVariable;
        swSketchSeg = (SketchSegment)vSketchSeg;
        if ((int)swSketchSegments_e.swSketchLINE == swSketchSeg.GetType())
        {
            swSketchCurrLine = (SketchLine)swSketchSeg;
            swSketchLineArr[swSketchLineArr.GetUpperBound(0)] = swSketchCurrLine;
            Array.Resize(ref swSketchLineArr, swSketchLineArr.GetUpperBound(0) + 2);
        }
    }
}

if (0 == swSketchLineArr.GetUpperBound(0))
{
    // No straight lines in this sketch
    functionReturnValue = null;
    return functionReturnValue;
}

// Remove last, empty sketch line
Array.Resize(ref swSketchLineArr, swSketchLineArr.GetUpperBound(0));
functionReturnValue = swSketchLineArr;
return functionReturnValue;
}

public bool GetSketchPoint(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchPoint swSketchPt)
{
    bool functionReturnValue = false;

    SketchPoint[] vSketchPtArr = null;
    vSketchPtArr = (SketchPoint[])swSketch.GetSketchPoints2();
    if ((vSketchPtArr != null))
    {
        // Use first point
        swSketchPt = (SketchPoint)vSketchPtArr[0];
        functionReturnValue = true;
        return functionReturnValue;
    }
    functionReturnValue = false;
    return functionReturnValue;
}

public bool FindVerticalOrigin(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchSegment swSketchSegVert,
SketchPoint swSketchPtVert)
{
    bool functionReturnValue = false;

    SketchLine[] vSketchLineArr = null;
    object vSketchLine = null;
    SketchLine swSketchCurrLine = default(SketchLine);
    SketchPoint swStartPt = default(SketchPoint);
    SketchPoint swEndPt = default(SketchPoint);

    // Get first vertical line
    vSketchLineArr = (SketchLine[])GetAllSketchLines(swApp, swModel, swSketch);

    if ((vSketchLineArr != null))
    {
        foreach (SketchLine vSketchLine_loopVariable in vSketchLineArr)
        {
            vSketchLine = vSketchLine_loopVariable;
            swSketchCurrLine = (SketchLine)vSketchLine;
            swStartPt = (SketchPoint)swSketchCurrLine.GetStartPoint2();
            swEndPt = (SketchPoint)swSketchCurrLine.GetEndPoint2();

            if (Math.Abs(swStartPt.X - swEndPt.X) < nTolerance)
            {
                swSketchSegVert = (SketchSegment)swSketchCurrLine;
                functionReturnValue = true;
                return functionReturnValue;
            }
        }
    }

    // Get first point
    functionReturnValue = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert);
    return functionReturnValue;
}

public bool FindHorizontalOrigin(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchSegment swSketchSegHoriz,
SketchPoint swSketchPtHoriz)
{
    bool functionReturnValue = false;

```

```

SketchLine[] vSketchLineArr = null;
object vSketchLine = null;
SketchLine swSketchCurrLine = default(SketchLine);
SketchPoint swStartPt = default(SketchPoint);
SketchPoint swEndPt = default(SketchPoint);

// Get first horizontal line
vSketchLineArr = (SketchLine[])GetAllSketchLines(swApp, swModel, swSketch);

if ((vSketchLineArr != null))
{
    foreach (SketchLine vSketchLine_loopVariable in vSketchLineArr)
    {
        vSketchLine = vSketchLine_loopVariable;
        swSketchCurrLine = (SketchLine)vSketchLine;
        swStartPt = (SketchPoint)swSketchCurrLine.GetStartPoint2();
        swEndPt = (SketchPoint)swSketchCurrLine.GetEndPoint2();

        if (Math.Abs(swStartPt.Y - swEndPt.Y) < nTolerance)
        {
            swSketchSegHoriz = (SketchSegment)swSketchCurrLine;
            functionReturnValue = true;
            return functionReturnValue;
        }
    }
}

// Get first point
functionReturnValue = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz);
return functionReturnValue;
}

public int AutoDimensionSketch(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SelectData swSelData)
{
    int functionReturnValue = 0;

    Feature swFeat = default(Feature);
    SketchSegment swSketchSegHoriz = null;
    SketchPoint swSketchPtHoriz = null;
    SketchSegment swSketchSegVert = null;
    SketchPoint swSketchPtVert = null;
    bool bRet = false;

    if (false == FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz))
    {
        functionReturnValue = (int)swAutodimStatus_e.swAutodimStatusDatumLineNotHorizontal;
        return functionReturnValue;
    }

    if (false == FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert))
    {
        functionReturnValue = (int)swAutodimStatus_e.swAutodimStatusDatumLineNotVertical;
        return functionReturnValue;
    }

    swFeat = (Feature)swSketch;

    bRet = swFeat.Select2(false, 0);

    // Editing sketch clears selections
    swModel.EditSketch();

    // Reselect sketch segments with correct marks for auto-dimensioning
    if ((swSketchSegVert != null))
    {
        // Vertical line is for horizontal datum
        bRet = swSketchSegVert.Select4(true, swSelData);
    }
    else if ((swSketchPtHoriz != null))
    {
        bRet = swSketchPtHoriz.Select4(true, swSelData);
    }
    else if ((swSketchPtVert != null))
    {
        // Use any sketch point for horizontal datum
        bRet = swSketchPtVert.Select4(true, swSelData);
    }

    if ((swSketchSegHoriz != null))
    {
        // Horizontal line is for vertical datum
        bRet = swSketchSegHoriz.Select4(true, swSelData);
    }
    else if ((swSketchPtVert != null))
    {
        bRet = swSketchPtVert.Select4(true, swSelData);
    }
    else if ((swSketchPtHoriz != null))
    {
        // Use any sketch point for vertical datum
        bRet = swSketchPtHoriz.Select4(true, swSelData);
    }

    // If contains circles use sketch points for datums

```

```

        if ((GetAllSketchLines(swApp, swModel, swSketch) == null))
        {
            if ((swSketchPtHoriz != null))
            {
                bRet = swSketchPtHoriz.Select4(false, swSelData);
            }
            else if ((swSketchPtVert != null))
            {
                bRet = swSketchPtVert.Select4(false, swSelData);
            }
        }

        swSketchMgr = swModel.SketchManager;
        nRetVal = swSketchMgr.FullyDefineSketch(true, true,
(int)swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical |
(int)swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, true, 1, null, 1, null, 1, 1);

        // Redraw so dimensions are displayed immediately
        double[] rect = null;
        rect = null;
        swModelView.GraphicsRedraw(rect);
        return functionReturnValue;
    }

    public SldWorks swApp;
}
}

```

Insert DXF/DWG File and Add Dimensions Example (VB.NET)

This example shows how to insert a DXF/DWG image on a preselected plane or face and then autodimension it.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Replace path_name with the pathname of an existing DXF/DWG file.
' 3. Select a plane or face on which to insert the DXF/DWG image.
'
' Postconditions:
' 1. Adds the DXF/DWG image as a sketch.
' 2. Autodimensions the sketch.
' 3. Use Zoom to Area to inspect the sketch dimensioning.
' 4. Press F5 to rebuild the model.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Const nTolerance As Double = 0.00000001
```

```
    Dim swModelView As ModelView
```

```
    Dim swSketchMgr As SketchManager
```

```
    Dim nRetVal As Long
```

```
    Function GetAllSketchLines(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch) As Object
```

```
        Dim vSketchSegArr As Object
```

```
        Dim vSketchSeg As Object
```

```
        Dim swSketchSeg As SketchSegment
```

```
        Dim swSketchCurrLine As SketchLine
```

```
        Dim swSketchLineArr() As SketchLine
```

```
        ReDim swSketchLineArr(0)
```

```
        vSketchSegArr = swSketch.GetSketchSegments
```

```
        If Not IsNothing(vSketchSegArr) Then
```

```
            For Each vSketchSeg In vSketchSegArr
```

```
                swSketchSeg = vSketchSeg
```

```
                If swSketchSegments_e.swSketchLINE = swSketchSeg.GetType Then
```

```
                    swSketchCurrLine = swSketchSeg
```

```
                    swSketchLineArr(UBound(swSketchLineArr)) = swSketchCurrLine
```

```
                    ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) + 1)
```

```
                End If
```

```
            Next
```

```
        End If
```

```
        If 0 = UBound(swSketchLineArr) Then
```

```
            ' No straight lines in this sketch
```

```
            GetAllSketchLines = Nothing
```

```
            Exit Function
```

```
        End If
```

```
        ' Remove last, empty sketch line
```

```
        ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) - 1)
```

```
        GetAllSketchLines = swSketchLineArr
```

```
    End Function
```

```
    Function GetSketchPoint(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSketchPt As SketchPoint) As Boolean
```

```
        Dim vSketchPtArr As Object
```

```
        vSketchPtArr = swSketch.GetSketchPoints2
```

```
        If Not IsNothing(vSketchPtArr) Then
```

```
            ' Use first point
```

```
            swSketchPt = vSketchPtArr(0)
```

```
            GetSketchPoint = True
```

```
            Exit Function
```

```
        End If
```

```
        GetSketchPoint = False
```

```
    End Function
```

```
    Function FindVerticalOrigin(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSketchSegVert As SketchSegment, ByVal swSketchPtVert As SketchPoint) As Boolean
```

```
        Dim vSketchLineArr As Object
```

```
        Dim vSketchLine As Object
```

```
        Dim swSketchCurrLine As SketchLine
```

```
        Dim swStartPt As SketchPoint
```

```
        Dim swEndPt As SketchPoint
```

```
        ' Get first vertical line
```

```
        vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)
```

```
        If Not IsNothing(vSketchLineArr) Then
```

```

        For Each vSketchLine In vSketchLineArr
            swSketchCurrLine = vSketchLine
            swStartPt = swSketchCurrLine.GetStartPoint2
            swEndPt = swSketchCurrLine.GetEndPoint2

            If Math.Abs(swStartPt.X - swEndPt.X) < nTolerance Then
                swSketchSegVert = swSketchCurrLine
                FindVerticalOrigin = True
                Exit Function
            End If
        Next
    End If

    ' Get first point
    FindVerticalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert)

End Function

Function FindHorizontalOrigin(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal
swSketchSegHoriz As SketchSegment, ByVal swSketchPtHoriz As SketchPoint) As Boolean

    Dim vSketchLineArr As Object
    Dim vSketchLine As Object
    Dim swSketchCurrLine As SketchLine
    Dim swStartPt As SketchPoint
    Dim swEndPt As SketchPoint

    ' Get first horizontal line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)

    If Not IsNothing(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            swSketchCurrLine = vSketchLine
            swStartPt = swSketchCurrLine.GetStartPoint2
            swEndPt = swSketchCurrLine.GetEndPoint2

            If Math.Abs(swStartPt.Y - swEndPt.Y) < nTolerance Then
                swSketchSegHoriz = swSketchCurrLine
                FindHorizontalOrigin = True
                Exit Function
            End If
        Next
    End If

    ' Get first point
    FindHorizontalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz)

End Function

Function AutoDimensionSketch(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSelData As
SelectData) As Long

    Dim swFeat As Feature
    Dim swSketchSegHoriz As SketchSegment = Nothing
    Dim swSketchPtHoriz As SketchPoint = Nothing
    Dim swSketchSegVert As SketchSegment = Nothing
    Dim swSketchPtVert As SketchPoint = Nothing
    Dim bRet As Boolean

    If False = FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz) Then
        AutoDimensionSketch = swAutodimStatus_e.swAutodimStatusDatumLineNotHorizontal
        Exit Function
    End If

    If False = FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert) Then
        AutoDimensionSketch = swAutodimStatus_e.swAutodimStatusDatumLineNotVertical
        Exit Function
    End If

    swFeat = swSketch

    bRet = swFeat.Select2(False, 0)

    ' Editing sketch clears selections
    swModel.EditSketch()

    ' Reselect sketch segments with correct marks for auto-dimensioning
    If Not swSketchSegVert Is Nothing Then
        ' Vertical line is for horizontal datum
        bRet = swSketchSegVert.Select4(True, swSelData)
    ElseIf Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(True, swSelData)
    ElseIf Not swSketchPtVert Is Nothing Then
        ' Use any sketch point for horizontal datum
        bRet = swSketchPtVert.Select4(True, swSelData)
    End If

    If Not swSketchSegHoriz Is Nothing Then
        ' Horizontal line is for vertical datum
        bRet = swSketchSegHoriz.Select4(True, swSelData)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(True, swSelData)
    ElseIf Not swSketchPtHoriz Is Nothing Then
        ' Use any sketch point for vertical datum
        bRet = swSketchPtHoriz.Select4(True, swSelData)
    End If

```

```

' No straight lines, probably contains circles
' so use sketch points for datums
If IsNothing(GetAllSketchLines(swApp, swModel, swSketch)) Then
    If Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(False, swSelData)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(False, swSelData)
    End If
End If

swSketchMgr = swModel.SketchManager
nRetVal = swSketchMgr.FullyDefineSketch(True, True,
swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical Or
swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, True, 1, Nothing, 1, Nothing, 1, 1)

' Redraw so dimensions are displayed immediately
Dim rect() As Double
rect = Nothing
swModelView.GraphicsRedraw(rect)

End Function

Sub main()

    Const sDwgFileName As String = "path_name"

    Dim swModel As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swSketch As Sketch
    Dim swSelMgr As SelectionMgr
    Dim swSelData As SelectData
    Dim bRet As Boolean
    Dim importData As ImportDxfDwgData

    swModel = swApp.ActiveDoc
    swModelView = swModel.ActiveView
    swFeatMgr = swModel.FeatureManager
    importData = swApp.GetImportFileData(sDwgFileName)

    'Unit
    importData.LengthUnit("") = swLengthUnit_e.swMM

    'Position
    bRet = importData.SetPosition("", swDwgImportEntitiesPositioning_e.swDwgEntitiesCentered, 0, 0)

    'Sheet scale
    bRet = importData.SetSheetScale("", 1.0#, 2.0#)

    'Paper size
    bRet = importData.SetPaperSize("", swDwgPaperSizes_e.swDwgPaperAsize, 0.0#, 0.0#)

    'Import method
    importData.ImportMethod("") = swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingPart

    swFeat = swFeatMgr.InsertDwg0rDxfFile2(sDwgFileName, importData)

    swSketch = swFeat.GetSpecificFeature2
    swSelMgr = swModel.SelectionManager
    swSelData = swSelMgr.CreateSelectData

    nRetVal = AutoDimensionSketch(swApp, swModel, swSketch, swSelData)

    Stop

    ' Rebuild to update sketch
    swModel.EditRebuild3()

End Sub

Public swApp As SldWorks

End Class

```

Insert Explode Line Sketch and Jog Line Example (C#)

This example shows how to insert a jog line in an explode line sketch, a type of 3D sketch.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\floxpress\ball_valve\ball_valve.sldasm.
//
// Postconditions:
// 1. Creates an exploded view of the assembly.
// 2. Adds a jog line, which is a type of explode line.
// 3. Examine the graphics area.
// 4. Locate 3DExplode1, the explode line sketch, on the
// ConfigurationManager tab (click the ConfigurationManager tab and
// expand default and ExplView1.)
//
// NOTE: Because this assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace InsertJogLineExplodeLine_CSharp.csproj
{
    partial
    class SolidWorksMacro
    {
        ModelDoc2 swModel;
        AssemblyDoc swAssembly;
        SketchManager swSketchMgr;
        SketchSegment swSketchSegment;

        public
        void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;
            swSketchMgr = swModel.SketchManager;

            //
            Explode the assembly
            swAssembly.AutoExplode();

            swModel.EditRebuild3();

            swModel.ViewZoomtofit2();

            //
            Insert an explode line sketch
            swSketchMgr.InsertExplodeLineSketch();

            //Create
            a line
            swSketchSegment = swSketchMgr.CreateLine(0, 0.1, 0, 0, 0.3, 0);

            swModel.ViewZoomtofit2();

            //
            Create a jog line using the line
            swSketchSegment.JogLine(0, 0.2, 0, 0.04, 0.25, 0);

            //
            Close the 3D sketch
            swSketchMgr.Insert3DSketch(true);
        }
        public SldWorks swApp;
    }
}
```

Insert Explode Line Sketch and Jog Line Example (VBA)

This example shows how to insert a jog line in an explode line sketch, a type of 3D sketch.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\floxpress\ball_valve\ball_valve.sldasm.  
'  
' Postconditions:  
' 1. Creates an exploded view of the assembly.  
' 2. Adds a jog line, which is a type of explode line.  
' 3. Examine the graphics area.  
' 4. Locate 3DExplode1, the explode line sketch, on the  
' ConfigurationManager tab (click the ConfigurationManager tab and  
' expand default and ExplView1.)  
'  
' NOTE: Because this assembly is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim SwApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim swSketchMgr As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
  
Sub main()  
  
    Set SwApp = Application.SldWorks  
    Set swModel = SwApp.ActiveDoc  
    Set swAssembly = swModel  
    Set swSketchMgr = swModel.SketchManager  
  
    ' Explode the assembly  
    swAssembly.AutoExplode  
  
    swModel.EditRebuild3  
  
    swModel.ViewZoomtofit2  
  
    ' Insert an explode line sketch  
    swSketchMgr.InsertExplodeLineSketch  
  
    ' Create a line  
    Set swSketchSegment = swSketchMgr.CreateLine(0, 0.1, 0, 0, 0.3, 0)  
  
    swModel.ViewZoomtofit2  
  
    ' Create a jog line using the line  
    swSketchSegment.JogLine 0, 0.2, 0, 0.04, 0.25, 0  
  
    ' Close the 3D sketch and rebuild  
    swSketchMgr.Insert3DSketch True  
  
End Sub
```

Insert Explode Line Sketch and Jog Line Example (VB.NET)

This example shows how to insert a jog line in an explode line sketch, a type of 3D sketch.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\floypress\ball_valve\ball_valve.sldasm.
'
' Postconditions:
' 1. Creates an exploded view of the assembly.
' 2. Adds a jog line, which is a type of explode line.
' 3. Examine the graphics area.
' 4. Locate 3DExplode1, the explode line sketch, on the
'    ConfigurationManager tab (click the ConfigurationManager tab and
'    expand default and ExplView1.)
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2
    Dim swAssembly As AssemblyDoc
    Dim swSketchMgr As SketchManager
    Dim swSketchSegment As SketchSegment

    Sub main()

        swModel = SwApp.ActiveDoc
        swAssembly = swModel
        swSketchMgr = swModel.SketchManager

        ' Explode the assembly
        swAssembly.AutoExplode()

        swModel.EditRebuild3()

        swModel.ViewZoomtofit2()

        ' Insert an explode line sketch
        swSketchMgr.InsertExplodeLineSketch()

        'Create a line
        swSketchSegment = swSketchMgr.CreateLine(0, 0.1, 0, 0, 0.3, 0)

        swModel.ViewZoomtofit2()

        ' Create a jog line using the line
        swSketchSegment.JogLine(0, 0.2, 0, 0.04, 0.25, 0)

        ' Close the 3D sketch
        swSketchMgr.Insert3DSketch(True)

    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Insert Explode Line Sketch and Route Line Example (C#)

This example shows how to insert a route line in an explode line sketch.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\floexpress\ball_valve\ball_valve.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates an exploded view of the assembly.
// 2. Adds a route line, which is a type of explode line.
// 3. Examine the Immediate window and graphics area.
// 4. Locate and click 3DExplode1, the explode line sketch, on the
// ConfigurationManager tab (click the ConfigurationManager
// tab and expand default and ExpView1).
//
// NOTE: Because this assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Sketch swSketch = default(Sketch);
            SketchManager swSketchMgr = default(SketchManager);
            Face2 swFace = default(Face2);
            object[] itemsToConnect = new object[2];
            object[] itemsReverse = new object[2];
            object[] itemsPath = new object[2];
            object[] alongXYZ = new object[2];
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;

            // Explode the assembly
            swAssembly.AutoExplode();
            swModel.EditRebuild3();
            swModel.ViewZoomToFit2();

            // Insert an explode line sketch
            swSketchMgr.InsertExplodeLineSketch();

            // Select two faces for the route line
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.00555234504082591, 0.0271707519863185, 0.00337956573349629, false, 0, null, 0);
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            itemsToConnect[0] = (object)swFace;
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00581777224675761, 0.0211322449790146, 0.127676153954326, true, 0, null, 0);
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            itemsToConnect[1] = (object)swFace;

            itemsReverse[0] = false;
            itemsReverse[1] = false;
            itemsPath[0] = false;
            itemsPath[1] = false;
            alongXYZ[0] = false;
            alongXYZ[1] = false;

            // Insert the route line in the explode line sketch
            swSketch = (Sketch)swModel.GetActiveSketch2();
            boolstatus = swSketch.InsertRouteLine((itemsToConnect), itemsReverse, itemsPath, alongXYZ);
            Debug.Print("Route line inserted in explode line sketch? " + boolstatus);

            // Close the explode line sketch
            swSketchMgr.InsertExplodeLineSketch();
        }
    }
}
```

Insert Explode Line Sketch and Route Line Example (VBA)

This example shows how to insert a route line in an explode line sketch.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\flopress\ball_valve\ball_valve.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates an exploded view of the assembly.
' 2. Adds a route line, which is a type of explode line.
' 3. Examine the Immediate window and graphics area.
' 4. Locate and click 3DExplode1, the explode line sketch, on the
'    ConfigurationManager tab (click the ConfigurationManager
'    tab and expand default and ExplView1).
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketch As SldWorks.Sketch
Dim swSketchMgr As SldWorks.SketchManager
Dim swFace As SldWorks.Face2
Dim itemsToConnect(1) As Variant
Dim itemsReverse(1) As Variant
Dim itemsPath(1) As Variant
Dim alongXYZ(1) As Variant
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager

    ' Explode the assembly
    swAssembly.AutoExplode

    swModel.EditRebuild3

    swModel.ViewZoomtofit2

    ' Insert an explode line sketch
    swSketchMgr.InsertExplodeLineSketch

    ' Select two faces for the route line
    boolstatus = swModelDocExt.SelectByID("", "FACE", -5.55234504082591E-03, 2.71707519863185E-02, 3.37956573349629E-03, False, 0, Nothing, 0)
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set itemsToConnect(0) = swFace

    boolstatus = swModelDocExt.SelectByID("", "FACE", 5.81777224675761E-03, 2.11322449790146E-02, 0.127676153954326, True, 0, Nothing, 0)
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set itemsToConnect(1) = swFace

    itemsReverse(0) = False
    itemsReverse(1) = False
    itemsPath(0) = False
    itemsPath(1) = False
    alongXYZ(0) = False
    alongXYZ(1) = False

    ' Insert the route line in the explode line sketch
    Set swSketch = swModel.GetActiveSketch2
    boolstatus = swSketch.InsertRouteLine(itemsToConnect, itemsReverse, itemsPath, alongXYZ)
    Debug.Print "Route line inserted in explode line sketch? " & boolstatus

    ' Close the explode line sketch
    swSketchMgr.InsertExplodeLineSketch

End Sub
```

Insert Explode Line Sketch and Route Line Example (VB.NET)

This example shows how to insert a route line in an explode line sketch.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\flopress\ball_valve\ball_valve.sldasm.
2. Open the Immediate window.

Postconditions:
1. Creates an exploded view of the assembly.
2. Adds a route line, which is a type of explode line.
3. Examine the Immediate window and graphics area.
4. Locate and click 3DExplode1, the explode line sketch, on the
   ConfigurationManager tab (click the ConfigurationManager
   tab and expand default and ExplView1).

NOTE: Because this assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssembly As AssemblyDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swSketch As Sketch
        Dim swSketchMgr As SketchManager
        Dim swFace As Face2
        Dim itemsToConnect(1) As Object
        Dim itemsReverse(1) As Object
        Dim itemsPath(1) As Object
        Dim alongXYZ(1) As Object
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swAssembly = swModel
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager

        ' Explode the assembly
        swAssembly.AutoExplode()
        swModel.EditRebuild3()
        swModel.ViewZoomtofit2()

        ' Insert an explode line sketch
        swSketchMgr.InsertExplodeLineSketch()

        ' Select two faces for the route line
        boolstatus = swModelDocExt.SelectByID("", "FACE", -0.00555234504082591, 0.0271707519863185, 0.00337956573349629, False, 0, Nothing, 0)
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        itemsToConnect(0) = swFace
        boolstatus = swModelDocExt.SelectByID("", "FACE", 0.00581777224675761, 0.0211322449790146, 0.127676153954326, True, 0, Nothing, 0)
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        itemsToConnect(1) = swFace

        itemsReverse(0) = False
        itemsReverse(1) = False
        itemsPath(0) = False
        itemsPath(1) = False
        alongXYZ(0) = False
        alongXYZ(1) = False

        ' Insert the route line in the explode line sketch
        swSketch = swModel.GetActiveSketch2
        boolstatus = swSketch.InsertRouteLine((itemsToConnect), itemsReverse, itemsPath, alongXYZ)
        Debug.Print("Route line inserted in explode line sketch? " & boolstatus)

        ' Close the explode line sketch
        swSketchMgr.InsertExplodeLineSketch()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Insert Extruded Surface Example (C#)

This example shows how to insert an extruded surface in a model.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new part and inserts Surface-Extrude1.
// 2. Expand the Surface Bodies folder to verify that it contains:
//    * Surface-Extrude[1]
//    * Surface-Extrude[2]
//    * Surface-Extrude[3]
// 3. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            object[] sketchLines = null;
            SketchSegment swSketchSegment = default(SketchSegment);
            SelectionMgr swSelMgr = default(SelectionMgr);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            SurfExtrudeFeatureData swSurfExtrudeFeature = default(SurfExtrudeFeatureData);
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0, 0);

            //Create sketches for extruded surface feature
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.03891024234798, 0.02968528649877, 0.0003646590412283, false, 0, null, 0);
            swModel.ClearSelection2(true);
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(-0.05517876768764, 0.008130204900836, 0, -0.02399076855985, -0.0155939995639, 0)
;
            swModel.ClearSelection2(true);
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(-0.003731897331531, 0.008130204900836, 0, 0.0285223581767, -0.02998846069981, 0)
;

            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.053579, 0.013995, 0.0, 0.06819, 0.018462, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModel.ShowNamedView2("Trimetric", 8);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);

            // Create a blind surface extrude
            // in two directions from the selected sketch
            // in a direction normal to the selected sketch plane
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeatureManager.FeatureExtrudeRefSurface3(false, false, (int)swStartConditions_e.swStartSketchPlane, 0, (int)swEndConditions_e.swEndCondBlind,
(int)swEndConditions_e.swEndCondBlind, 0.01, 0.01, true, false,
false, false, 0.4, 0, false, false, false, false, false, false,
false, false);
            swModel.ClearSelection2(true);

            // Get Surface-Extrude1 feature
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            status = swModelDocExt.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSurfExtrudeFeature = (SurfExtrudeFeatureData)swFeature.GetDefinition();

            //Access Surface-Extrude1 feature data
            swSurfExtrudeFeature.AccessSelections(swModel, null);

            Debug.Print(swFeature.Name);
            Debug.Print("    Depth:");
            Debug.Print("    Forward direction: " + swSurfExtrudeFeature.GetDepth(true));
            Debug.Print("    Reverse direction: " + swSurfExtrudeFeature.GetDepth(false));
            Debug.Print("    End condition as defined in swSurfaceExtendEndCond_e:");
            Debug.Print("    Forward direction: " + swSurfExtrudeFeature.GetEndCondition(true));
            Debug.Print("    Reverse direction: " + swSurfExtrudeFeature.GetEndCondition(false));
            Debug.Print("    Reverse offset enabled:");
            Debug.Print("    Forward direction? " + swSurfExtrudeFeature.GetReverseOffset(true));
            Debug.Print("    Reverse direction? " + swSurfExtrudeFeature.GetReverseOffset(false));
            Debug.Print("    Translate surface setting enabled:");
            Debug.Print("    Forward direction? " + swSurfExtrudeFeature.GetTranslateSurface(true));
            Debug.Print("    Reverse direction? " + swSurfExtrudeFeature.GetTranslateSurface(false));
            Debug.Print("    Surface extruded in both directions? " + swSurfExtrudeFeature.BothDirections);
            Debug.Print("    Extrusion reversed? " + swSurfExtrudeFeature.ReverseDirection);
            Debug.Print("    Direction 1 end:");
            Debug.Print("    Capped? " + swSurfExtrudeFeature.D1CapEnd);
            Debug.Print("    Drafted? " + swSurfExtrudeFeature.D1DraftOn);
            if (swSurfExtrudeFeature.D1DraftOn)
            {
                Debug.Print("    Angle: " + swSurfExtrudeFeature.D1DraftAngle);
                Debug.Print("    Inward (false) or outward (true)? " + swSurfExtrudeFeature.D1DraftOutward);
            }
            Debug.Print("    Direction 2 end:");
            Debug.Print("    Capped? " + swSurfExtrudeFeature.D2CapEnd);
            Debug.Print("    Drafted? " + swSurfExtrudeFeature.D2DraftOn);
            if (swSurfExtrudeFeature.D2DraftOn)
            {
                Debug.Print("    Angle: " + swSurfExtrudeFeature.D2DraftAngle);
                Debug.Print("    Inward (false) or outward (true)? " + swSurfExtrudeFeature.D2DraftOutward);
            }
        }
    }
}
```

```
    }
    Debug.Print(" Delete original face? " + swSurfExtrudeFeature.DeleteOriginalFace);
    Debug.Print(" Knit extrusion result? " + swSurfExtrudeFeature.KnitResult);

    //Release Surface-Extrude1 feature data
    swSurfExtrudeFeature.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Insert Extruded Surface Example (VBA)

This example shows how to insert an extruded surface in a model.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part and inserts Surface-Extrude1.
' 2. Expand the Surface Bodies folder to verify that it contains:
'    * Surface-Extrude[1]
'    * Surface-Extrude[2]
'    * Surface-Extrude[3]
' 3. Examine the Immediate window and graphics area.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim sketchLines As Variant
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swSurfExtrudeFeature As SldWorks.SurfExtrudeFeatureData
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)

    'Create sketches for extruded surface feature
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.03891024234798, 0.02968528649877, 3.646590412283E-04, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    sketchLines = swSketchManager.CreateCornerRectangle(-0.05517876768764, 0.008130204900836, 0, -0.02399076855985, -0.0155939995639, 0)
    swModel.ClearSelection2 True
    sketchLines = swSketchManager.CreateCornerRectangle(-0.003731897331531, 0.008130204900836, 0, 0.0285223581767, -0.02998846069981, 0)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateCircle(0.053579, 0.013995, 0#, 0.06819, 0.018462, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True
    swModel.ShowNamedView2 "Trimetric", 8
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    ' Create a blind surface extrude
    ' in two directions from the selected sketch
    ' in a direction normal to the selected sketch plane
    Set swFeatureManager = swModel.FeatureManager
    swFeatureManager.FeatureExtruRefSurface3 False, False, swStartConditions_e.swStartSketchPlane, 0, swEndConditions_e.swEndCondBlind,
    swEndConditions_e.swEndCondBlind, 0.01, 0.01, True, False, False, False, 0.4, 0, False, False, False, False, False, False, False, False

    swModel.ClearSelection2 True

    ' Get Surface-Extrude1 feature
    Set swSelMgr = swModel.SelectionManager
    status = swModelDocExt.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swSurfExtrudeFeature = swFeature.GetDefinition

    'Access Surface-Extrude1 feature data
    swSurfExtrudeFeature.AccessSelections swModel, Nothing

    Debug.Print swFeature.Name
    Debug.Print "    Depth:"
    Debug.Print "    Forward direction: " & swSurfExtrudeFeature.GetDepth(True)
    Debug.Print "    Reverse direction: " & swSurfExtrudeFeature.GetDepth(False)
    Debug.Print "    End condition as defined in swSurfaceExtendEndCond_e:"
    Debug.Print "    Forward direction: " & swSurfExtrudeFeature.GetEndCondition(True)
    Debug.Print "    Reverse direction: " & swSurfExtrudeFeature.GetEndCondition(False)
    Debug.Print "    Reverse offset enabled:"
    Debug.Print "    Forward direction? " & swSurfExtrudeFeature.GetReverseOffset(True)
    Debug.Print "    Reverse direction? " & swSurfExtrudeFeature.GetReverseOffset(False)
    Debug.Print "    Translate surface setting enabled:"
    Debug.Print "    Forward direction? " & swSurfExtrudeFeature.GetTranslateSurface(True)
    Debug.Print "    Reverse direction? " & swSurfExtrudeFeature.GetTranslateSurface(False)
    Debug.Print "    Surface extruded in both directions? " & swSurfExtrudeFeature.BothDirections
    Debug.Print "    Extrusion reversed? " & swSurfExtrudeFeature.ReverseDirection
    Debug.Print "    Direction 1 end:"
    Debug.Print "    Capped? " & swSurfExtrudeFeature.D1CapEnd
    Debug.Print "    Drafted? " & swSurfExtrudeFeature.D1DraftOn
    If swSurfExtrudeFeature.D1DraftOn Then
        Debug.Print "    Angle: " & swSurfExtrudeFeature.D1DraftAngle
        Debug.Print "    Inward (false) or outward (true)? " & swSurfExtrudeFeature.D1DraftOutward
    End If
    Debug.Print "    Direction 2 end:"
    Debug.Print "    Capped? " & swSurfExtrudeFeature.D2CapEnd
    Debug.Print "    Drafted? " & swSurfExtrudeFeature.D2DraftOn

```

```
If swSurfExtrudeFeature.D2DraftOn Then
    Debug.Print "    Angle: " & swSurfExtrudeFeature.D2DraftAngle
    Debug.Print "    Inward (false) or outward (true)? " & swSurfExtrudeFeature.D2DraftOutward
End If
Debug.Print "    Delete original face? " & swSurfExtrudeFeature.DeleteOriginalFace
Debug.Print "    Knit extrusion result? " & swSurfExtrudeFeature.KnitResult

'Release Surface-Extrude1 feature data
swSurfExtrudeFeature.ReleaseSelectionAccess

End Sub
```

Insert Extruded Surface Example (VB.NET)

This example shows how to insert an extruded surface in a model.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Creates a new part and inserts Surface-Extrude1.
2. Expand the Surface Bodies folder to verify that it contains:
   * Surface-Extrude[1]
   * Surface-Extrude[2]
   * Surface-Extrude[3]
3. Examine the Immediate window and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim sketchLines As Object
        Dim swSketchSegment As SketchSegment
        Dim swSelMgr As SelectionMgr
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swSurfExtrudeFeature As SurfExtrudeFeatureData
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)

        'Create sketches for extruded surface feature
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", -0.03891024234798, 0.02968528649877, 0.0003646590412283, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        sketchLines = swSketchManager.CreateCornerRectangle(-0.05517876768764, 0.008130204900836, 0, -0.02399076855985, -0.0155939995639, 0)
        swModel.ClearSelection2(True)
        sketchLines = swSketchManager.CreateCornerRectangle(-0.003731897331531, 0.008130204900836, 0, 0.0285223581767, -0.02998846069981, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateCircle(0.053579, 0.013995, 0.0#, 0.06819, 0.018462, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)
        swModel.ShowNamedView2("Trimetric", 8)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

        ' Create a blind surface extrude
        ' in two directions from the selected sketch
        ' in a direction normal to the selected sketch plane
        swFeatureManager = swModel.FeatureManager
        swFeatureManager.FeatureExtrudeRefSurface3(False, False, swStartConditions_e.swStartSketchPlane, 0, swEndConditions_e.swEndCondBlind, swEndCondition
s_e.swEndCondBlind, 0.01, 0.01, True, False, False, False, 0.4, 0, False, False, False, False, False, False, False, False)
        swModel.ClearSelection2(True)

        ' Get Surface-Extrude1 feature
        swSelMgr = swModel.SelectionManager
        status = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swSurfExtrudeFeature = swFeature.GetDefinition

        'Access Surface-Extrude1 feature data
        swSurfExtrudeFeature.AccessSelections(swModel, Nothing)

        Debug.Print(swFeature.Name)
        Debug.Print(" Depth:")
        Debug.Print(" Forward direction: " & swSurfExtrudeFeature.GetDepth(True))
        Debug.Print(" Reverse direction: " & swSurfExtrudeFeature.GetDepth(False))
        Debug.Print(" End condition as defined in swSurfaceExtendEndCond_e:")
        Debug.Print(" Forward direction: " & swSurfExtrudeFeature.GetEndCondition(True))
        Debug.Print(" Reverse direction: " & swSurfExtrudeFeature.GetEndCondition(False))
        Debug.Print(" Reverse offset enabled:")
        Debug.Print(" Forward direction? " & swSurfExtrudeFeature.GetReverseOffset(True))
        Debug.Print(" Reverse direction? " & swSurfExtrudeFeature.GetReverseOffset(False))
        Debug.Print(" Translate surface setting enabled:")
        Debug.Print(" Forward direction? " & swSurfExtrudeFeature.GetTranslateSurface(True))
        Debug.Print(" Reverse direction? " & swSurfExtrudeFeature.GetTranslateSurface(False))
        Debug.Print(" Surface extruded in both directions? " & swSurfExtrudeFeature.BothDirections)
        Debug.Print(" Extrusion reversed? " & swSurfExtrudeFeature.ReverseDirection)
        Debug.Print(" Direction 1 end:")
        Debug.Print(" Capped? " & swSurfExtrudeFeature.D1CapEnd)
        Debug.Print(" Drafted? " & swSurfExtrudeFeature.D1DraftOn)
        If swSurfExtrudeFeature.D1DraftOn Then
            Debug.Print(" Angle: " & swSurfExtrudeFeature.D1DraftAngle)
            Debug.Print(" Inward (false) or outward (true)? " & swSurfExtrudeFeature.D1DraftOutward)
        End If
        Debug.Print(" Direction 2 end:")
        Debug.Print(" Capped? " & swSurfExtrudeFeature.D2CapEnd)
        Debug.Print(" Drafted? " & swSurfExtrudeFeature.D2DraftOn)
        If swSurfExtrudeFeature.D2DraftOn Then
            Debug.Print(" Angle: " & swSurfExtrudeFeature.D2DraftAngle)
            Debug.Print(" Inward (false) or outward (true)? " & swSurfExtrudeFeature.D2DraftOutward)
        End If
        Debug.Print(" Delete original face? " & swSurfExtrudeFeature.DeleteOriginalFace)
        Debug.Print(" Knit extrusion result? " & swSurfExtrudeFeature.KnitResult)

        'Release Surface-Extrude1 feature data
        swSurfExtrudeFeature.ReleaseSelectionAccess()
    End Sub

End Class

```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Create Extrusion Feature Example (C#)

This example shows how to create an extrusion feature.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates a Boss-Extrude1 feature.
// 2. Examine the FeatureManager design tree and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace CreateExtrusionFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        Feature myFeature;
        ModelDoc2 Part;
        RefPlane myRefPlane;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            myRefPlane = (RefPlane)Part.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0);
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            myRefPlane = (RefPlane)Part.FeatureManager.InsertRefPlane(8, 0.02, 0, 0, 0, 0);

            boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 0, 0, 0, false, 0, null, 0);
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0250462141853123, 0.0157487558892494, 0, 0.0275128867944718,
-0.015559011842391, 0);

            Part.SketchManager.InsertSketch(true);

            // Sketch to extrude
            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            // Start condition reference
            boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 0.00105020593408751, -0.00195369982668282,
0.0248175428318827, true, 32, null, 0);
            // End condition reference
            boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", 0.0068370744701368, -0.004419862088339, 0.018892268568016,
true, 1, null, 0);

            // Boss extrusion start condition reference is Plane2, and the extrusion end is offset 3 mm from the end condition
            reference, Plane1
            myFeature = (Feature)Part.FeatureManager.FeatureExtrusion3(true, false, true,
(int)swEndConditions_e.swEndCondOffsetFromSurface, 0, 0.003, 0.003, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
(int)swStartConditions_e.swStartSurface, 0, false);

        }

        public SldWorks swApp;
    }
}
```

Create Extrusion Feature Example (VBA)

This example shows how to create an extrusion feature.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates a Boss-Extrude1 feature.
' 2. Examine the FeatureManager design tree and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim myFeature As SldWorks.Feature
Dim Part As SldWorks.ModelDoc2
Dim myRefPlane As SldWorks.RefPlane
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set Part = swApp.ActiveDoc

    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.02, 0, 0, 0, 0)

    boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-2.50462141853123E-02, 1.57487558892494E-02, 0, 2.75128867944718E-02,
-0.015559011842391, 0)

    Part.SketchManager.InsertSketch True

    ' Sketch to extrude
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    ' Start condition reference
    boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 1.05020593408751E-03, -1.95369982668282E-03, 2.48175428318827E-02,
True, 32, Nothing, 0)
    ' End condition reference
    boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", 6.8370744701368E-03, -0.004419862088339, 0.018892268568016, True, 1,
Nothing, 0)

    ' Boss extrusion start condition reference is Plane2, and the extrusion end is offset 3 mm from the end condition reference,
Plane1
    Set myFeature = Part.FeatureManager.FeatureExtrusion3(True, False, True, swEndCondOffsetFromSurface, 0, 0.003, 0.003, False,
False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, swStartSurface, 0,
False)

End Sub
```

Create Extrusion Feature Example (VB.NET)

This example shows how to create an extrusion feature.

```
'-----
' Preconditions: Verify that the specified part template exists.
```

```
' Postconditions:
' 1. Creates a Boss-Extrude1 feature.
' 2. Examine the FeatureManager design tree and graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim myFeature As Feature
    Dim Part As ModelDoc2
    Dim myRefPlane As RefPlane
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.01, 0, 0, 0, 0)
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.02, 0, 0, 0, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Dim vSkLines As Object
```

```
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0250462141853123, 0.0157487558892494, 0, 0.0275128867944718,
-0.015559011842391, 0)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        ' Sketch to extrude
```

```
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
        ' Start condition reference
```

```
        boolstatus = Part.Extension.SelectByID2("Plane2", "PLANE", 0.00105020593408751, -0.00195369982668282, 0.0248175428318827,
True, 32, Nothing, 0)
```

```
        ' End condition reference
```

```
        boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", 0.0068370744701368, -0.004419862088339, 0.018892268568016, True,
1, Nothing, 0)
```

```
        ' Boss extrusion start condition reference is Plane2, and the extrusion end is offset 3 mm from the end condition
reference, Plane1
```

```
        myFeature = Part.FeatureManager.FeatureExtrusion3(True, False, True, swEndConditions_e.swEndCondOffsetFromSurface, 0,
0.003, 0.003, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True,
swStartConditions_e.swStartSurface, 0, False)
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Insert Fill-surface Feature Example (C#)

This example shows how to insert a fill-surface feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Sketches a circle on the Front Plane.
// 2. Offsets the Front Plane to create Plane1.
// 3. Sketches a circle on Plane1.
// 4. Creates a thin-feature loft using the circles
//    created in steps 1 and 3.
// 5. Selects one of the sketches to use for
//    the fill-surface feature.
// 6. Creates a fill-surface feature named Surface-Fill1.
// 7. Gets, sets, and prints some properties of the fill-surface feature
//    to the Immediate window.
// 8. Examine the FeatureManager design, graphics area, and the
//    Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace InsertFillFeature2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatMgr = default(FeatureManager);
            RefPlane swRefPlane = default(RefPlane);
            Feature swFeat = default(Feature);
            FillSurfaceFeatureData swFillSurfaceFeatureData = default(FillSurfaceFeatureData);
            object selObj = null;
            bool status = false;
            int nbrPatchEntities = 0;
            object[] patchEntities = null;
            object[] entTypes = null;
            int[] entTypesArray;
            int i = 0;

            //Open a new model document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\part.prtdot", (int)swDwgPaperSizes_e.swDwgPaperASize, 0, 0);

            //Select the front plane and sketch a circle
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swModel.ClearSelection2(true);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.018863, -0.048015, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Offset the front plane to create Plane1
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            swRefPlane = (RefPlane)swFeatMgr.InsertRefPlane(0, 0.0762, 0, 0, 0, 0);
            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.005144, -0.017148, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Create a loft as a thin feature
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -0.0120997659765269, 0.0131954647737917, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", -0.0137458916138411, 0.0497220981864567, 0, true, 1, null, 0);
            swFeatMgr.InsertProtrusionBlend2(false, true, false, 1, 0, 0, 1, 1, true, true, true, 0.000254, 0.000254, 0, true, true, true, (int)swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge);

            //Get the sketch for the fill-surface feature
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -0.0309259362651374, -0.0150632202505945, 0.0265529245975468, true, 257, null, (int)swSelectOption_e.swSelectOptionDefault);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            selObj = (object)swSelMgr.GetSelectedObject6(1, 257);

            //Insert the fill-surface feature
            swFeat = (Feature)swFeatMgr.InsertFillSurface2(2, (int)swFeatureFillSurfaceOptions_e.swOptimizeSurface, selObj, (int)swContactType_e.swContactNone, null, null);

            //Access the fill-surface feature
            swFillSurfaceFeatureData = (FillSurfaceFeatureData)swFeat.GetDefinition();
            status = swFillSurfaceFeatureData.AccessSelections(swModel, null);
            Debug.Print("Fill-surface feature: ");
            Debug.Print(" Number of constraint curves: " + swFillSurfaceFeatureData.GetConstraintCurvesCount());
            nbrPatchEntities = swFillSurfaceFeatureData.GetPatchBoundaryCount();
            Debug.Print(" Number of entities used to define the patch boundary: " + nbrPatchEntities);
            if (nbrPatchEntities > 0)
            {
                //Get the type of patch boundary entities
                patchEntities = (object[])swFillSurfaceFeatureData.GetPatchBoundary(out entTypes);
                entTypesArray = (int[])entTypes;
                for (i = 0; i <= nbrPatchEntities - 1; i++)
                {
                    Debug.Print(" Type of entity for patch boundary " + (i + 1) + " (1 = edge; 9 = sketch) : " + entTypesArray[i]);
                }
            }
        }
    }
}
```

```

        Debug.Print(" Results merged? " + swFillSurfaceFeatureData.Merge);
        swFillSurfaceFeatureData.OptimizeSurface = false;
        Debug.Print(" Patch optimized? " + swFillSurfaceFeatureData.OptimizeSurface);
        Debug.Print(" Original resolution control: " + swFillSurfaceFeatureData.ResolutionControl);
        if (swFillSurfaceFeatureData.OptimizeSurface == false)
        {
            swFillSurfaceFeatureData.ResolutionControl = 1;
        }
        Debug.Print(" Updated resolution control (valid values 1, 2 and 3; setting this value only valid if patch is not optimized): " + swFillSurfaceFeatureData.ResolutionControl);
    }
    status = swFeat.ModifyDefinition(swFillSurfaceFeatureData, swModel, null);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Insert Fill-surface Feature Example (VBA)

This example shows how to insert a fill-surface feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sketches a circle on the Front Plane.
' 2. Offsets the Front Plane to create Plane1.
' 3. Sketches a circle on Plane1.
' 4. Creates a thin-feature loft using the circles
'    created in steps 1 and 3.
' 5. Selects one of the sketches to use for
'    the fill-surface feature.
' 6. Creates a fill-surface feature named Surface-Fill1.
' 7. Gets, sets, and prints some properties of the fill-surface feature
'    to the Immediate window.
' 8. Examine the FeatureManager design, graphics area, and the
'    the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatMgr As SldWorks.FeatureManager
Dim swRefPlane As SldWorks.RefPlane
Dim swFeat As SldWorks.Feature
Dim swFillSurfaceFeatureData As SldWorks.FillSurfaceFeatureData
Dim selObj As Object
Dim status As Boolean
Dim nbrPatchEntities As Long
Dim patchEntities As Variant
Dim entTypes As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open a new model document
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\part.prtdot", swDwgPaperAsize, 0, 0)

    'Select the front plane and sketch a circle
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    swModel.ClearSelection2 True
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.018863, -0.048015, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Offset the front plane to create Plane1
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatMgr = swModel.FeatureManager
    Set swRefPlane = swFeatMgr.InsertRefPlane(8, 0.0762, 0, 0, 0, 0)
    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.005144, -0.017148, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create a loft as a thin feature
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -1.20997659765269E-02, 1.31954647737917E-02, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", -1.37458916138411E-02, 4.97220981864567E-02, 0, True, 1, Nothing, 0)
    swFeatMgr.InsertProtrusionBlend2 False, True, False, 1, 0, 0, 1, 1, True, True, True, 0.000254, 0.000254, 0, True, True, True,
    swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge

    'Get the sketch for the fill-surface feature
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -3.09259362651374E-02, -1.50632202505945E-02, 2.65529245975468E-02, True, 257, Nothing,
    swSelectOptionDefault)
    Set swSelMgr = swModel.SelectionManager
    Set selObj = swSelMgr.GetSelectedObject6(1, 257)

    'Insert the fill-surface feature
    Set swFeat = swFeatMgr.InsertFillSurface2(2, swOptimizeSurface, selObj, swContact, Nothing, Nothing)

    'Access the fill-surface feature
    Set swFillSurfaceFeatureData = swFeat.GetDefinition
    status = swFillSurfaceFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Fill-surface feature: "
    Debug.Print "  Number of constraint curves: " & swFillSurfaceFeatureData.GetConstraintCurvesCount
    nbrPatchEntities = swFillSurfaceFeatureData.GetPatchBoundaryCount
    Debug.Print "  Number of entities used to define the patch boundary: " & nbrPatchEntities
    If nbrPatchEntities > 0 Then
        'Get the type of patch boundary entities
        patchEntities = swFillSurfaceFeatureData.GetPatchBoundary(entTypes)
        For i = 0 To nbrPatchEntities - 1
            Debug.Print ("  Type of entity for patch boundary " & (i + 1) & " (1 = edge; 9 = sketch) : " & entTypes(i))
        Next i
        Debug.Print "  Results merged? " & swFillSurfaceFeatureData.Merge
        swFillSurfaceFeatureData.OptimizeSurface = False
    End If
End Sub

```

```
Debug.Print " Patch optimized? " & swFillSurfaceFeatureData.OptimizeSurface
Debug.Print " Original resolution control: " & swFillSurfaceFeatureData.ResolutionControl
If swFillSurfaceFeatureData.OptimizeSurface = False Then
    swFillSurfaceFeatureData.ResolutionControl = 1
End If
Debug.Print " Updated resolution control (valid values 1, 2 and 3; setting this value only valid if patch is not optimized): " &
swFillSurfaceFeatureData.ResolutionControl
End If
status = swFeat.ModifyDefinition(swFillSurfaceFeatureData, swModel, Nothing)

End Sub
```

Insert Fill-surface Feature Example (VB.NET)

This example shows how to insert a fill-surface feature.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Sketches a circle on the Front Plane.
2. Offsets the Front Plane to create Plane1.
3. Sketches a circle on Plane1.
4. Creates a thin-feature loft using the circles
   created in steps 1 and 3.
5. Selects one of the sketches to use for
   the fill-surface feature.
6. Creates a fill-surface feature named Surface-Fill1.
7. Gets, sets, and prints some properties of the fill-surface feature
   to the Immediate window.
8. Examine the FeatureManager design, graphics area, and the
   the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatMgr As FeatureManager
        Dim swRefPlane As RefPlane
        Dim swFeat As Feature
        Dim swFillSurfaceFeatureData As FillSurfaceFeatureData
        Dim selObj As Object
        Dim status As Boolean
        Dim nbrPatchEntities As Integer
        Dim patchEntities() As Object
        Dim entTypes As Object = Nothing
        Dim i As Integer

        'Open a new model document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\part.prt", swDwgPaperSizes_e.swDwgPaperASize, 0, 0)

        'Select the front plane and sketch a circle
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swModel.ClearSelection2(True)
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.018863, -0.048015, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Offset the front plane to create Plane1
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swFeatMgr = swModel.FeatureManager
        swRefPlane = swFeatMgr.InsertRefPlane(0, 0.0762, 0, 0, 0, 0)
        status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.005144, -0.017148, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Create a loft as a thin feature
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", -0.0120997659765269, 0.0131954647737917, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", -0.0137458916138411, 0.0497220981864567, 0, True, 1, Nothing, 0)
        swFeatMgr.InsertProtrusionBlend2(False, True, False, 1, 0, 0, 1, 1, True, True, 0.000254, 0.000254, 0, True, True, True,
        swGuideCurveInfluence_e.swGuideCurveInfluenceNextEdge)

        'Get the sketch for the fill-surface feature
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", -0.0309259362651374, -0.0150632202505945, 0.0265529245975468, True, 257, Nothing, swSelect
Option_e.swSelectOptionDefault)
        swSelMgr = swModel.SelectionManager
        selObj = swSelMgr.GetSelectedObject6(1, 257)
        'Insert the fill-surface feature
        swFeat = swFeatMgr.InsertFillSurface2(2, swFeatureFillSurfaceOptions_e.swOptimizeSurface, selObj, swContactType_e.swContact, Nothing, Nothing)

        'Access the fill-surface feature
        swFillSurfaceFeatureData = swFeat.GetDefinition
        status = swFillSurfaceFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Fill-surface feature: ")
        Debug.Print(" Number of constraint curves: " & swFillSurfaceFeatureData.GetConstraintCurvesCount)
        nbrPatchEntities = swFillSurfaceFeatureData.GetPatchBoundaryCount
        Debug.Print(" Number of entities used to define the patch boundary: " & nbrPatchEntities)
        If nbrPatchEntities > 0 Then
            'Get the type of patch boundary entities
            patchEntities = swFillSurfaceFeatureData.GetPatchBoundary(entTypes)
            For i = 0 To nbrPatchEntities - 1
                Debug.Print(" Type of entity for patch boundary " & (i + 1) & " (1 = edge; 9 = sketch) : " & entTypes(i))
            Next i
            Debug.Print(" Results merged? " & swFillSurfaceFeatureData.Merge)
            swFillSurfaceFeatureData.OptimizeSurface = False
            Debug.Print(" Patch optimized? " & swFillSurfaceFeatureData.OptimizeSurface)
            Debug.Print(" Original resolution control: " & swFillSurfaceFeatureData.ResolutionControl)
            If swFillSurfaceFeatureData.OptimizeSurface = False Then
                swFillSurfaceFeatureData.ResolutionControl = 1
            End If
            Debug.Print(" Updated resolution control (valid values 1, 2 and 3; setting this value only valid if patch is not optimized): " & swFillSurfac
eFeatureData.ResolutionControl)
        End If
        status = swFeat.ModifyDefinition(swFillSurfaceFeatureData, swModel, Nothing)

    End Sub

```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Insert Folder in FeatureManager Design Tree Example (VBA)

This examples shows how to insert a new folder in the FeatureManager design tree above the selected folder. T

```
'-----
' Precondition:
' 1. Open a model document.
' 2. Select the folder above which you want to
'    insert a new folder.
'
' Postconditions:
' 1. Inserts a new folder in the FeatureManager
'    design tree above the selected folder. The
'    selected folder is now a sub-folder to the
'    new folder, Folder1_NEW.
' 2. Expand Folder1_NEW to verify step 1.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swFeat As SldWorks.Feature
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swFeat = swFeatMgr.InsertFeatureTreeFolder2(swFeatureTreeFolder_Containing)

    ' Return Nothing if unable to create the folder
    If Not swFeat Is Nothing Then
        swFeat.Name = swFeat.Name & "_NEW"
    End If

End Sub
```

Insert Forming Tool Feature Example (C#)

This example shows how to insert a forming tool feature into a sheet metal part.

```
//-----
// Preconditions:
// 1. Open a sheet metal part.
// 2. Verify that the specified forming tool part exists.
// 3. Select a face on which to apply the counter sink emboss forming tool from
//    the Design Library.
//
// Postconditions:
// 1. Inserts the counter sink emboss1 feature.
// 2. Examine the FeatureManager design tree and graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        string formingTool;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            // Insert a counter sink emboss forming tool feature
            formingTool = "C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2025\\design library\\forming tools\\embosses\\counter sink
emboss.sldprt";
            myFeature = Part.FeatureManager.InsertFormToolFeature(formingTool, false, 0.0, "", true, true, true, true, false);
        }

        public SldWorks swApp;
    }
}
```

Insert Forming Tool Feature Example (VBA)

This example shows how to insert a forming tool feature into a sheet metal part.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Verify that the specified forming tool part exists.
' 3. Select a face on which to apply the counter sink emboss forming tool from
'    the Design Library.
'
' Postconditions:
' 1. Inserts the counter sink emboss1 feature.
' 2. Examine the FeatureManager design tree and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myFeature As SldWorks.Feature
Dim formingTool As String

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    ' Insert a counter sink emboss forming tool feature
    formingTool = "C:\ProgramData\SolidWorks\SOLIDWORKS 2025\design library\forming tools\embosses\counter sink emboss.sldprt"
    Set myFeature = Part.FeatureManager.InsertFormToolFeature(formingTool, False, 0#, "", True, True, True, True, False)

End Sub
```

Insert Forming Tool Feature Example (VB.NET)

This example shows how to insert a forming tool feature into a sheet metal part.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Verify that the specified forming tool part exists.
' 3. Select a face on which to apply the counter sink emboss forming tool from
'    the Design Library.
'
' Postconditions:
' 1. Inserts the counter sink emboss1 feature.
' 2. Examine the FeatureManager design tree and graphics area.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim myFeature As Feature

    Dim formingTool as String

    Sub main()
        Part = swApp.ActiveDoc

        ' Insert a counter sink emboss
        forming tool feature

        formingTool = "C:\ProgramData\SolidWorks\SOLIDWORKS
2025\design library\forming tools\embosses\counter sink emboss.sldprt"

        myFeature = Part.FeatureManager.InsertFormToolFeature(formingTool, False,
0.0#, "", True, True, True, True, False)

    End Sub

    Public swApp As SldWorks

End Class
```

Insert Free Point Curve Feature Example (C#)

This example shows how to insert a free point curve feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Create c:\temp, if this folder does not exist.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new part document.
// 2. Inserts a free point curve feature.
// 3. Gets some free point curve feature data.
// 4. Saves the free point curve feature's points to a file.
// 5. Examine the graphics area, FeatureManager design tree,
//    Immediate window, and c:\temp.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FreePointCurveFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            FreePointCurveFeatureData swFreePointCurveFeatureData = default(FreePointCurveFeatureData);
            bool status = false;
            int nbrPoints = 0;
            double[] pointArray = null;
            int i = 0;

            //Create new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);

            //Insert free point curve feature
            swModel.InsertCurveFileBegin();
            status = swModel.InsertCurveFilePoint(0, 0, 0);
            status = swModel.InsertCurveFilePoint(0, 0, 0.0127);
            status = swModel.InsertCurveFilePoint(0, 0, 0.0254);
            status = swModel.InsertCurveFilePoint(0, 0, 0.0381);
            status = swModel.InsertCurveFilePoint(0, 0.0254, 0.0381);
            status = swModel.InsertCurveFilePoint(0, 0.0381, 0.0381);
            status = swModel.InsertCurveFileEnd();

            //Get free point curve feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            //Get number of points in free point curve feature
            swFreePointCurveFeatureData = (FreePointCurveFeatureData)swFeature.GetDefinition();
            nbrPoints = swFreePointCurveFeatureData.GetPointCount();
            Debug.Print("Number of points in free point curve feature: " + nbrPoints);
            //Get the points in free point curve feature
            pointArray = (double[])swFreePointCurveFeatureData.PointArray;
            Debug.Print("Points in free point curve feature: ");
            for (i = 0; i <= nbrPoints - 1; i++)
            {
                Debug.Print(" " + pointArray[i]);
            }
            //Save the points in free point curve feature to file
            status = swFreePointCurveFeatureData.SavePointsToFile("c:\\temp\\MyFreePointCurveFeature.sldcrv");
            Debug.Print("Curve file created in specified folder: " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert Free Point Curve Feature Example (VBA)

This example shows how to insert a free point curve feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Create c:\temp, if this folder does not exist.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document.
' 2. Inserts a free point curve feature.
' 3. Gets some free point curve feature data.
' 4. Saves the free point curve feature's points to a file.
' 5. Examine the graphics area, FeatureManager design tree,
'    Immediate window, and c:\temp.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swFreePointCurveFeatureData As SldWorks.FreePointCurveFeatureData
Dim status As Boolean
Dim nbrPoints As Long
Dim pointArray As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Create new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

    'Insert free point curve feature
    swModel.InsertCurveFileBegin
    status = swModel.InsertCurveFilePoint(0, 0, 0)
    status = swModel.InsertCurveFilePoint(0, 0, 0.0127)
    status = swModel.InsertCurveFilePoint(0, 0, 0.0254)
    status = swModel.InsertCurveFilePoint(0, 0, 0.0381)
    status = swModel.InsertCurveFilePoint(0, 0.0254, 0.0381)
    status = swModel.InsertCurveFilePoint(0, 0.0381, 0.0381)
    status = swModel.InsertCurveFileEnd()

    'Get free point curve feature
    Set swModelDocExt = swModel.Extension
    Set swSelectionMgr = swModel.SelectionManager
    status = swModelDocExt.SelectByID("Curve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    'Get number of points in free point curve feature
    Set swFreePointCurveFeatureData = swFeature.GetDefinition
    nbrPoints = swFreePointCurveFeatureData.GetPointCount
    Debug.Print ("Number of points in free point curve feature: " & nbrPoints)
    'Get the points in free point curve feature
    pointArray = swFreePointCurveFeatureData.pointArray
    Debug.Print "Points in free point curve feature: "
    For i = 0 To nbrPoints - 1
        Debug.Print " " & pointArray(i)
    Next i
    'Save the points in free point curve feature to file
    status = swFreePointCurveFeatureData.SavePointsToFile("c:\temp\MyFreePointCurveFeature.sldcrv")
    Debug.Print "Curve file created in specified folder: " & status

End Sub
```

Insert Free Point Curve Feature Example (VB.NET)

This example shows how to insert a free point curve feature.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Create c:\temp, if this folder does not exist.
3. Open the Immediate window.
-----
Postconditions:
1. Creates a new part document.
2. Inserts a free point curve feature.
3. Gets some free point curve feature data.
4. Saves the free point curve feature's points to a file.
5. Examine the graphics area, FeatureManager design tree,
   Immediate window, and c:\temp.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swFreePointCurveFeatureData As FreePointCurveFeatureData
        Dim status As Boolean
        Dim nbrPoints As Integer
        Dim pointArray() As Double
        Dim i As Integer

        'Create new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

        'Insert free point curve feature
        swModel.InsertCurveFileBegin()
        status = swModel.InsertCurveFilePoint(0, 0, 0)
        status = swModel.InsertCurveFilePoint(0, 0, 0.0127)
        status = swModel.InsertCurveFilePoint(0, 0, 0.0254)
        status = swModel.InsertCurveFilePoint(0, 0, 0.0381)
        status = swModel.InsertCurveFilePoint(0, 0.0254, 0.0381)
        status = swModel.InsertCurveFilePoint(0, 0.0381, 0.0381)
        status = swModel.InsertCurveFileEnd()

        'Get free point curve feature
        swModelDocExt = swModel.Extension
        swSelectionMgr = swModel.SelectionManager
        status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        'Get number of points in free point curve feature
        swFreePointCurveFeatureData = swFeature.GetDefinition
        nbrPoints = swFreePointCurveFeatureData.GetPointCount
        Debug.Print("Number of points in free point curve feature: " & nbrPoints)
        'Get the points in free point curve feature
        pointArray = swFreePointCurveFeatureData.PointArray
        Debug.Print("Points in free point curve feature: ")
        For i = 0 To nbrPoints - 1
            Debug.Print(" " & pointArray(i))
        Next i
        'Save the points in free point curve feature to file
        status = swFreePointCurveFeatureData.SavePointsToFile("c:\temp\MyFreePointCurveFeature.sldcrv")
        Debug.Print("Curve file created in specified folder: " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Insert General Table (VBA)

This example shows how to insert a general table in a drawing document and how to hide and show a row in that table.

```
'-----  
' Preconditions: Drawing document is open.  
'  
' Postconditions: The third row of the general table is  
'                 hidden and shown.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swTable As SldWorks.TableAnnotation  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swDrawing = swModel  
  
' Insert general table with four rows  
Set swTable = swDrawing.InsertTableAnnotation2(True, 0, 0, swBOMConfigurationAnchor_TopLeft, "", 4, 2)  
Debug.Print "Nbr of rows before hiding third third row: " & swTable.RowCount  
' Hide the third row  
swTable.RowHidden(2) = True  
Debug.Print "Nbr of rows after hiding third row: " & swTable.RowCount  
' Show the third row  
swTable.RowHidden(2) = False  
Debug.Print "Nbr of rows after showing third row: " & swTable.RowCount  
  
End Sub
```

Insert General Table in Part Example (C#)

This example shows how to insert a general table annotation into a model document.

```
//-----  
// Preconditions: Open a part.  
//  
// Postconditions:  
// 1. Inserts a general table in the part.  
// 2. Examine the graphics area.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
namespace InsertGeneralTable_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 Part;  
        ModelDocExtension modelDocExt;  
  
        TableAnnotation myTable;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.ActiveDoc;  
            modelDocExt = Part.Extension;  
            myTable = modelDocExt.InsertGeneralTableAnnotation(true, 0, 0,  
(int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2);  
  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Insert General Table in Part Example (VBA)

This example shows how to insert a general table annotation into a model document.

```
'-----  
' Preconditions: Open a part.  
'  
' Postconditions:  
' 1. Inserts a general table in the part.  
' 2. Examine the graphics area.  
'-----  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim myTable As SldWorks.TableAnnotation  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set modelDocExt = Part.Extension  
    Set myTable = modelDocExt.InsertGeneralTableAnnotation(True, 0, 0, swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
  
End Sub
```

Insert General Table in Part Example (VB.NET)

This example shows how to insert a general table annotation into a model document.

```
'-----  
' Preconditions: Open a part.  
'-----  
' Postconditions:  
' 1. Inserts a general table in the part.  
' 2. Examine the graphics area.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Dim Part As ModelDoc2  
    Dim modelDocExt As ModelDocExtension  
    Dim myTable As TableAnnotation  
  
    Sub main()  
  
        Part = swApp.ActiveDoc  
        modelDocExt = Part.Extension  
        myTable = modelDocExt.InsertGeneralTableAnnotation(True, 0, 0,  
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Insert General Tolerance Table Example (C#)

This example shows how to create a general tolerance table annotation.

```
//-----
// Preconditions:
// 1. Open a part.
// 2. Ensure that the specified table template exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Creates a general tolerance table annotation.
// 2. Inspect the Immediate window, the graphics area, and the Tables folder
//    of the FeatureManager design tree.
//-----

using SolidWorks.Interop.sldworks;
using System.Diagnostics;

namespace InsertTable_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            GeneralToleranceTableAnnotation gttAnno = default(GeneralToleranceTableAnnotation);
            GeneralToleranceTableFeature gttFeat = default(GeneralToleranceTableFeature);
            Feature feat = default(Feature);
            ModelDocExtension mde = default(ModelDocExtension);
            bool boolstatus = false;

            Part = (ModelDoc2)swApp.ActiveDoc;
            mde = Part.Extension;
            gttAnno = mde.InsertGeneralToleranceTableAnnotation("c:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-
standard.sldbomtbt", 481, 163);
            gttFeat = gttAnno.GeneralToleranceTableFeature;
            feat = gttFeat.GetFeature();
            Debug.Print("Created " + feat.Name);
            boolstatus = Part.EditRebuild3();
        }

        // The SldWorks swApp variable is pre-assigned for you.
        public SldWorks swApp;
    }
}
```

Insert General Tolerance Table Example (VBA)

This example shows how to create a general tolerance table annotation.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Ensure that the specified table template exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a general tolerance table annotation.  
' 2. Inspect the Immediate window, the graphics area, and the Tables folder  
'    of the FeatureManager design tree.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim gttAnno As SldWorks.GeneralToleranceTableAnnotation  
Dim gttFeat As SldWorks.GeneralToleranceTableFeature  
Dim feat As SldWorks.Feature  
Dim mde As SldWorks.ModelDocExtension  
Dim boolstatus As Boolean  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set mde = Part.Extension  
    Set gttAnno = mde.InsertGeneralToleranceTableAnnotation("c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-  
standard.sldbomtbt", 481, 163)  
    Set gttFeat = gttAnno.GeneralToleranceTableFeature  
    Set feat = gttFeat.GetFeature  
    Debug.Print "Created " & feat.Name  
    boolstatus = Part.EditRebuild3()  
  
End Sub
```

Insert General Tolerance Table Example (VB.NET)

This example shows how to create a general tolerance table annotation.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Ensure that the specified table template exists.  
' 3. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Creates a general tolerance table annotation.  
' 2. Inspect the Immediate window, the graphics area, and the Tables folder  
'    of the FeatureManager design tree.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim Part As ModelDoc2  
        Dim gttAnno As GeneralToleranceTableAnnotation  
        Dim gttFeat As GeneralToleranceTableFeature  
        Dim feat As Feature  
        Dim mde As ModelDocExtension  
        Dim boolstatus As Boolean
```

```
        Part = swApp.ActiveDoc  
        mde = Part.Extension  
        gttAnno = mde.InsertGeneralToleranceTableAnnotation("c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-  
standard.sldbomtbt", 481, 163)  
        gttFeat = gttAnno.GeneralToleranceTableFeature  
        feat = gttFeat.GetFeature  
        Debug.Print("Created " & feat.Name)  
        boolstatus = Part.EditRebuild3()
```

```
    End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Insert Grid System Feature Example (C#)

This example shows how to insert a Grid System feature into a model.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a sketch of a circle.
// 3. Inserts GridSystem1, which contains Surface-Extrude1, Level1, Level2,
//    and two derived sketches, in the FeatureManager design tree.
// 4. Sets the height of each level of the surface extrude to 50.0 mm.
// 5. Examine the Immediate window and graphics area.
// 6. Expand GridSystem1 in FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace InsertGridFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        Feature GridFeature;
        ModelDoc2 Part;
        ModelView myModelView;
        SketchSegment skSegment;
        bool boolstatus;
        int longstatus;

        public void Main()
        {
            swApp.ResetUntitledCount(0, 0, 0);
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prt", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            myModelView = (ModelView)Part.ActiveView;
            myModelView.FrameLeft = 0;
            myModelView.FrameTop = 0;
            myModelView.FrameState = (int)swWindowState_e.swWindowMaximized;
            Part.ShowNamedView2("Isometric", 7);

            skSegment = Part.SketchManager.CreateCircle(-0.019633, 0.076084, 0.0, -0.001997, 0.081475, 0.0);
            Part.SketchManager.InsertSketch(true);

            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);

            object offsetArray = null;
            double[] heightsArray = new double[2];
            heightsArray[0] = 0.05;
            heightsArray[1] = 0.05;
            offsetArray = heightsArray;

            GridFeature = Part.FeatureManager.InsertGridFeature((offsetArray));
            Debug.Print(GridFeature.Name + " was created.");

            Part.ViewZoomtofit();
        }
    }
}
public SldWorks swApp;
```

Insert Grid System Feature Example (VBA)

This example shows how to insert a Grid System feature into a model.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch of a circle.
' 3. Inserts GridSystem1, which contains Surface-Extrude1, Level1, Level2,
'    and two derived sketches, in the FeatureManager design tree.
' 4. Sets the height of each level of the surface extrude to 50.0 mm.
' 5. Examine the Immediate window and graphics area.
' 6. Expand GridSystem1 in FeatureManager design tree.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim GridFeature As SldWorks.Feature
Dim Part As SldWorks.ModelDoc2
Dim myModelView As SldWorks.ModelView
Dim skSegment As SldWorks.SketchSegment
Dim boolstatus As Boolean
Dim longstatus As Long

Sub main()
    Set swApp = Application.SldWorks
    swApp.ResetUntitledCount 0, 0, 0
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2011\templates\Part.prt", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set Part = swApp.ActiveDoc

    Set myModelView = Part.ActiveView
    myModelView.FrameLeft = 0
    myModelView.FrameTop = 0
    myModelView.FrameState = swWindowState_e.swWindowStateMaximized
    Part.ShowNamedView2 "Isometric", 7

    Set skSegment = Part.SketchManager.CreateCircle(-0.019633, 0.076084, 0#, -0.001997, 0.081475, 0#)
    Part.SketchManager.InsertSketch True

    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    Dim offsetArray As Variant
    Dim heightsArray() As Double
    ReDim heightsArray(0 To 1) As Double
    heightsArray(0) = 0.05
    heightsArray(1) = 0.05
    offsetArray = heightsArray

    Set GridFeature = Part.FeatureManager.InsertGridFeature(offsetArray)
    Debug.Print GridFeature.Name & " was created."

    Part.ViewZoomtofit
End Sub
```

Insert Grid System Feature Example (VB.NET)

This example shows how to insert a Grid System Feature into a model.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch of a circle.
' 3. Inserts GridSystem1, which contains Surface-Extrude1, Level1, Level2,
'    and two derived sketches, in the FeatureManager design tree.
' 4. Sets the height of each level of the surface extrude to 50.0 mm.
' 5. Examine the Immediate window and graphics area.
' 6. Expand GridSystem1 in FeatureManager design tree.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim GridFeature As Feature
```

```
    Dim Part As ModelDoc2
```

```
    Dim myModelView As ModelView
```

```
    Dim skSegment As SketchSegment
```

```
    Dim boolstatus As Boolean
```

```
    Dim longstatus As Long
```

```
    Sub main()
```

```
        swApp.ResetUntitledCount(0, 0, 0)
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
```

```
        swApp.ActivateDoc2("Part1", False, longstatus)
```

```
        Part = swApp.ActiveDoc
```

```
        myModelView = Part.ActiveView
```

```
        myModelView.FrameLeft = 0
```

```
        myModelView.FrameTop = 0
```

```
        myModelView.FrameState = swWindowState_e.swWindowMaximized
```

```
        Part.ShowNamedView2("**Isometric", 7)
```

```
        skSegment = Part.SketchManager.CreateCircle(-0.019633, 0.076084, 0.0#, -0.001997, 0.081475, 0.0#)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Dim offsetArray As Object
```

```
        Dim heightsArray(1) As Double
```

```
        heightsArray(0) = 0.05
```

```
        heightsArray(1) = 0.05
```

```
        offsetArray = heightsArray
```

```
        GridFeature = Part.FeatureManager.InsertGridFeature((offsetArray))
```

```
        Debug.Print(GridFeature.Name & " was created.")
```

```
        Part.ViewZoomtofit()
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Insert and Activate Ground Plane Example (VBA)

This example shows how to insert and activate a ground plane in the current configuration of an assembly.

```
'-----  
' Preconditions: Open public_documents\introsw\vanity_assembly.sldasm.  
'  
' Postconditions:  
' 1. Inserts Ground Plane1 in the Ground Planes folder.  
' 2. Inspect the FeatureManager design tree.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim featMgr As SldWorks.FeatureManager  
Dim feat As SldWorks.Feature  
Dim groundPlanes As Variant  
Dim boolstatus As Boolean  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set featMgr = Part.FeatureManager  
  
    boolstatus = Part.Extension.SelectByRay(-9.07572786993001E-02, -3.35232970080597E-02, -0.329520078481409, -0.560777860996441,  
0.738723196597798, -0.373920084275486, 1.28607075733318E-02, 2, False, 0, 0)  
    Set feat = featMgr.InsertGroundPlane(False)  
  
    boolstatus = Part.Extension.SelectByID2("Ground Plane1", "MAGNETICGROUNDPLANE", 0, 0, 0, False, 0, Nothing, 0)  
    boolstatus = Part.ActivateGroundPlane(swThisConfiguration, Null)  
    groundPlanes = Part.GetActiveGroundPlane(swAllConfiguration, Null)  
  
End Sub
```

Insert GTol Example (C#)

This example shows how to insert a GTol with a bent leader.

```
//-----
// Preconditions:
// 1. Verify that the drawing exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Inserts a GTol.
// 3. Sets and gets the length of the bent leader.
// 4. Gets whether to display the projected tolerance zone and its height
//    in the GTol.
// 5. Examine the Immediate window and GTol.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDrawing = default(DrawingDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            GTol swGtol = default(GTol);
            Annotation swAnno = default(Annotation);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int frameNbr = 0;
            int tolNbr = 0;
            bool display = false;
            string height = "";

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            swDrawing = (DrawingDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
            swApp.ActivateDoc3("foodprocessor - Sheet1", false, (int)swRebuildOnActivation_e.swRebuildActiveDoc, ref errors);
            swModel = (ModelDoc2)swDrawing;

            swGtol = (GTol)swModel.InsertGtol();
            if ((swGtol != null))
            {
                swGtol.SetFrameSymbols2(1, "<IGTOL-POSI>", false, "", false, "", "", "", "");
                swGtol.SetFrameValues2(1, "0.2", "", "B-A-C<MOD-MMC>", "B<MOD-MMC>-C<MOD-LMC>", "C<MOD-MMC>-A");
                swGtol.SetPTZHeight2(1, 1, true, "50");
                swGtol.SetBetweenTwoPoints(false, "", "");
                swAnno = (Annotation)swGtol.GetAnnotation();
                if ((swAnno != null))
                {
                    status = swAnno.SetPosition(0.319315975204224, 0.12666668401487, 0);
                    errors = swAnno.SetLeader3((int)swLeaderStyle_e.swBENT, (int)swLeaderSide_e.swLS_LEFT, true, false, false, false);
                    swAnno.BentLeaderLength = 0.05;
                    Debug.Print("Bent leader length: " + swAnno.BentLeaderLength);
                }
            }

            frameNbr = 1;
            tolNbr = 1;
            height = "";
            status = swGtol.GetPTZHeight2(frameNbr, tolNbr, out display, out height);
            Debug.Print("Projected tolerance zone: ");
            Debug.Print("  Display: " + display);
            Debug.Print("  Height: " + height);

            swModel.WindowRedraw();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert GTol Example (VBA)

This example shows how to insert a GTol with a bent leader.

```
'-----
' Preconditions:
' 1. Verify that the drawing exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Inserts a GTol.
' 3. Sets and gets the length of the bent leader.
' 4. Gets whether to display the projected tolerance zone and its height
'    in the GTol.
' 5. Examine the Immediate window and GTol.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swDrawing As SldWorks.DrawingDoc
Dim swModel As SldWorks.ModelDoc2
Dim swGtol As SldWorks.Gtol
Dim swAnno As SldWorks.Annotation
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim frameNbr As Long
Dim tolNbr As Long
Dim display As Boolean
Dim height As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    swApp.ActivateDoc3 "foodprocessor - Sheet1", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors
    Set swModel = swDrawing

    Set swGtol = swModel.InsertGtol()
    If Not swGtol Is Nothing Then
        swGtol.SetFrameSymbols2 1, "<IGTOL-POSI>", False, "", False, "", "", "", ""
        status = swGtol.SetFrameValues2(1, "0.4", "", "B-A-C<MOD-MMC>", "B<MOD-MMC>-C<MOD-LMC>", "C<MOD-MMC>-A")
        status = swGtol.SetPTZHeight2(1, 1, True, "50")
        swGtol.SetBetweenTwoPoints False, "", ""
        Set swAnno = swGtol.GetAnnotation()
        If Not swAnno Is Nothing Then
            status = swAnno.SetPosition(0.319315975204224, 0.12666668401487, 0)
            errors = swAnno.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_LEFT, True, False, False, False)
            swAnno.BentLeaderLength = 0.05
            Debug.Print "Bent leader length: " & swAnno.BentLeaderLength
        End If
    End If

    frameNbr = 1
    tolNbr = 1
    height = ""
    status = swGtol.GetPTZHeight2(frameNbr, tolNbr, display, height)
    Debug.Print "Projected tolerance zone: "
    Debug.Print "    Display: " & display
    Debug.Print "    Height: " & height

    swModel.WindowRedraw

End Sub
```